

Advanced Encryption Standard

5

TÓPICOS ABORDADOS

- 5.1 ARITMÉTICA DE CORPO FINITO**
- 5.2 ESTRUTURA DO AES**
 - Estrutura geral
 - Estrutura detalhada
- 5.3 FUNÇÕES DE TRANSFORMAÇÃO DO AES**
 - Transformação subBytes
 - Transformação ShiftRows
 - Transformação MixColumns
 - Transformação AddRoundKey
- 5.4 EXPANSÃO DE CHAVE DO AES**
 - Algoritmo de expansão de chave
 - Raciocínio
- 5.5 EXEMPLO DE AES**
 - Resultados
 - Efeito avalanche
- 5.6 IMPLEMENTAÇÃO DO AES**
 - Cifra inversa equivalente
 - Aspectos de implementação
- 5.7 LEITURA RECOMENDADA**
- 5.8 PRINCIPAIS TERMOS, PERGUNTAS PARA REVISÃO E PROBLEMAS**
- APÊNDICE 5A** POLINÔMIOS COM COEFICIENTES EM $GF(2^8)$
- APÊNDICE 5B** AES SIMPLIFICADO

OBJETIVOS DE APRENDIZAGEM

APÓS ESTUDAR ESTE CAPÍTULO, VOCÊ SERÁ CAPAZ DE:

- ☒ Apresentar de forma geral a estrutura do Advanced Encryption Standard, ou AES.
- ☒ Compreender as quatro transformações utilizadas no AES.
- ☒ Explicar o algoritmo de expansão de chave do AES.
- ☒ Compreender o uso de polinômios com coeficientes em $GF(2^8)$.

"Parece muito simples."

"Eu tenho resolvido outras cifras de uma obtusidade dez mil vezes maior. As circunstâncias, e uma certa preferência mental, levaram-me a ter interesse em tais enigmas, e é bem possível duvidar se a engenhosidade humana é capaz de construir um enigma do tipo que a ingenuidade não consiga, através de uma aplicação adequada, resolver."

— O Escaravelho de Ouro, Edgar Allan Poe

O Advanced Encryption Standard (Padrão de Encriptação Avançada), ou AES, foi publicado pelo National Institute of Standards and Technology (Instituto Nacional de Padrões e Tecnologia), ou NIST, em 2001. O AES é uma cifra simétrica de bloco que pretende substituir o DES como o padrão para uma grande variedade de aplicações. Comparada às cifras de chave pública como a RSA, a estrutura do AES e a maioria das cifras simétricas são bastante complexas e não explicadas tão facilmente quanto outras cifras criptográficas. Consequentemente, o leitor poderá começar com uma versão simplificada do AES, que é descrita no Apêndice 5B (em <sv.pearson.com.br>, em inglês). Essa versão permite que o leitor simule a encriptação e a decrptação manualmente e adquira uma boa compreensão do funcionamento dos detalhes do algoritmo. A experiência em sala de aula indica que um estudo dessa versão simplificada aumenta a compreensão do AES.¹ Um possível método de estudo é ler o capítulo inteiro, em seguida ver o Apêndice 5B com bastante atenção e, então, reler a parte principal do capítulo.

O Apêndice H (em <sv.pearson.com.br>, em inglês) aborda o critério de avaliação usado pelo NIST a fim de selecionar os candidatos para o AES, além do raciocínio para a escolha do Rijndael, que foi o candidato vencedor. Esse material é útil para compreender não apenas o projeto do AES, mas também os critérios para avaliar quaisquer algoritmos simétricos de encriptação.

5.1 ARITMÉTICA DE CORPO FINITO

No AES, todas as operações são realizadas em 8 bits. Em particular, as operações aritméticas de soma, multiplicação e divisão são feitas sobre o corpo finito $GF(2^8)$. A Seção 4.7 discute essas operações com mais detalhes. Para o leitor que não tenha estudado o Capítulo 4 e como uma revisão rápida para aqueles que o tenham feito, esta seção resume os conceitos importantes.

Basicamente, um corpo é um conjunto no qual nós podemos somar, subtrair, multiplicar e dividir sem sair dele. A divisão é definida com a seguinte regra: $a/b = a(b^{-1})$. Um exemplo de um corpo finito (aquele que possui um número finito de elementos) é o conjunto Z_p que contém todos os inteiros $\{0, 1, \dots, p-1\}$, onde p é um número primo e cujo cálculo é feito módulo p .

Praticamente todos os algoritmos de encriptação, tantos simétricos quanto aqueles de chave pública, envolvem operações aritméticas com inteiros. Se uma das operações usadas no algoritmo for uma divisão, então precisaremos trabalhar em aritmética definida sobre um corpo; isso é decorrente de a divisão exigir que cada elemento diferente de zero tenha um inverso multiplicativo. Por conveniência e para melhorar a eficiência da implementação, poderíamos trabalhar com inteiros que caibam exatamente em um número de bits, sem desperdício de padrões. Ou seja, queremos trabalhar com números inteiros na faixa de 0 a $2^n - 1$ que se encaixam em uma *word* de n bits. Infelizmente, o conjunto Z_{2^n} composto por esses números inteiros, usando aritmética modular, não é um corpo. Por exemplo, o inteiro 2 não possui multiplicador inverso em Z_{2^n} , ou seja, não existe um inteiro b , de modo que $2b \bmod 2^n = 1$.

Existe uma maneira de definir um corpo finito contendo 2^n elementos, sendo esse corpo denominado $GF(2^n)$. Considere o conjunto S de todos os polinômios de grau $n-1$ ou menor com coeficientes binários. Então, cada polinômio possui a forma

$$f(x) = a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \dots + a_1x + a_0 = \sum_{i=0}^{n-1} a_i x^i$$

onde cada a_i assume o valor 0 ou 1. Existe um total de 2^n polinômios diferentes em S . Se $n = 3$, os $2^3 = 8$ polinômios no conjunto são

$$\begin{array}{cccccc} 0 & x & x^2 & x^2 & x & \\ 1 & x & 1 & x^2 & 1 & x^2 & x & 1 \end{array}$$

Com a definição apropriada das operações aritméticas, cada conjunto S é um corpo finito. A determinação consiste dos seguintes elementos:

1. A aritmética segue as regras comuns da aritmética polinomial usando as diretrizes básicas da álgebra com os dois refinamentos a seguir.

¹ No entanto, o leitor poderá tranquilamente pular o Apêndice 5B, ou no máximo fazer uma leitura sem se deter muito nos detalhes. Caso se sinta perdido ou preso aos detalhes do AES, poderá voltar atrás e estudar o AES simplificado.

2. A aritmética nos coeficientes é calculada usando módulo 2. Isso é o mesmo que realizar a operação XOR.
3. Se a multiplicação resulta em um polinômio de grau maior que $n - 1$, então ele é reduzido ao módulo de algum polinômio irredutível $m(x)$ de grau n . Isso significa que dividimos por $m(x)$ e mantemos o resto. Para o polinômio $f(x)$, o resto é expresso como $r(x) = f(x) \bmod m(x)$. Um polinômio $m(x)$ é chamado de **irredutível** se, e somente se, $m(x)$ não puder ser expresso como um produto de dois polinômios, ambos de grau menor que o de $m(x)$.

Por exemplo, para construir o corpo finito $GF(2^3)$, precisamos escolher um polinômio irredutível de grau 3. Existem somente dois polinômios como esse: $(x^3 + x^2 + 1)$ e $(x^3 + x + 1)$. A adição é equivalente a usar o XOR dos termos de mesmo grau. Dessa forma, $(x + 1) + x = 1$.

Um polinômio em $GF(2^n)$ pode ser representado de forma única pelos seus n coeficientes binários $(a_{n-1}a_{n-2} \dots a_0)$. Portanto, todo polinômio em $GF(2^n)$ consegue ser representado por um número de n bits. A adição é realizada através do XOR bit a bit, dos dois elementos de n bits. Não existe uma operação tão simples quanto o XOR que faça a multiplicação em $GF(2^n)$. No entanto, há uma técnica razoavelmente direta e de fácil implementação. Em linhas gerais, pode ser demonstrado que a multiplicação de um número em $GF(2^n)$ por 2 consiste de um deslocamento (*shift*) para a esquerda seguido de um XOR condicional com uma constante. A multiplicação por números maiores pode ser realizada pela aplicação dessa regra repetidamente.

Por exemplo, o AES usa aritmética de um corpo finito $GF(2^8)$ com o polinômio irredutível $m(x) = x^8 + x^4 + x^3 + x + 1$. Considere dois elementos $A = (a_7a_6 \dots a_1a_0)$ e $B = (b_7b_6 \dots b_1b_0)$. A soma é $A + B = (c_7c_6 \dots c_1c_0)$, onde $c_i = a_i \oplus b_i$. A multiplicação $\{02\} \cdot A$ é igual a $(a_6 \dots a_1a_00)$ se $a_7 = 0$ e é igual a $(a_6 \dots a_1a_00) \oplus (00011011)$ se $a_7 = 1$.²

Resumindo, o AES opera sobre bytes de 8 bits. A adição de dois bytes é definida como uma operação XOR bit a bit. A multiplicação de dois bytes é definida como aquela no corpo finito $GF(2^8)$, com o polinômio irredutível³ $m(x) = x^8 + x^4 + x^3 + x + 1$. Os desenvolvedores do Rijndael indicam, como sua motivação para selecionar esse dentre os 30 polinômios irredutíveis possíveis de grau 8, que ele é o primeiro na lista dada em [LIDL94].

5.2 ESTRUTURA DO AES

Estrutura geral

A Figura 5.1 mostra a estrutura geral do processo de encriptação do AES. A cifra recebe como entrada um bloco de texto sem formatação de tamanho 128 bits, ou 16 bytes. O comprimento da chave pode ser 16, 24 ou 32 bytes (128, 192 ou 256 bits). O algoritmo é denominado AES-128, AES-192 ou AES-256, dependendo do tamanho da chave.

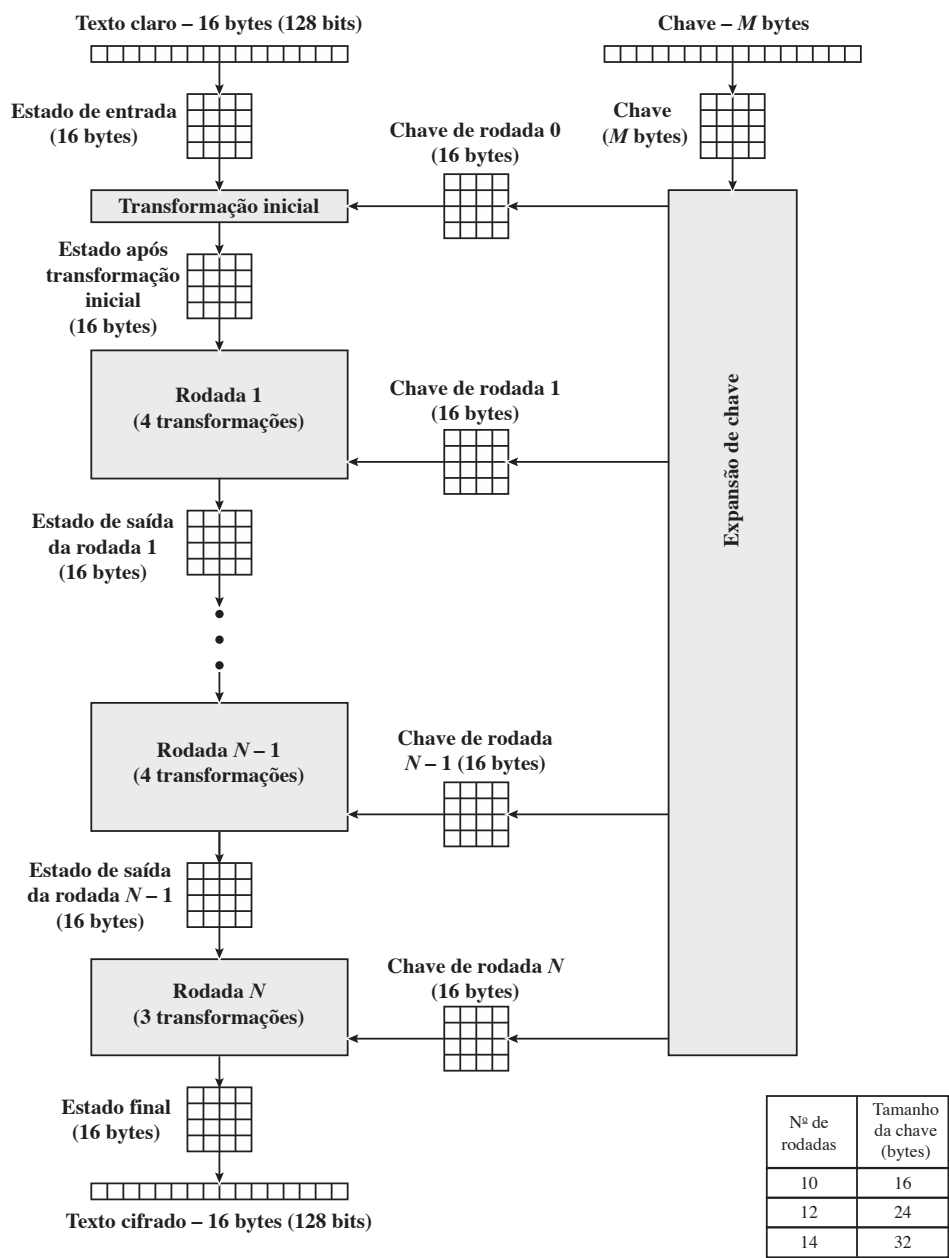
A entrada para os algoritmos de encriptação e decríptação é um único bloco de 128 bits. No FIPS PUB 197, esse bloco é indicado como uma matriz quadrada de bytes 4×4 . Esse bloco é copiado para um array **Estado**, que é modificado a cada etapa de encriptação ou decríptação. Após a etapa final, **Estado** é copiado para uma matriz de saída. Essas operações são descritas na Figura 5.2a. De modo semelhante, a chave é apresentada como uma matriz quadrada de bytes. Essa chave é, então, expandida para um conjunto de palavras de chave. A Figura 5.2b mostra a expansão para a chave de 128 bits. Cada palavra tem quatro bytes, e o conjunto total é de 44 palavras, para a chave de 128 bits. Observe que a ordenação de bytes dentro de uma matriz de chaves é por coluna. Assim, por exemplo, os primeiros quatro bytes de entrada de texto claro de 128 bits para a cifra de encriptação ocupam a primeira coluna da matriz **in**, os próximos quatro bytes ocupam a segunda coluna, e assim por diante. Da mesma forma, os primeiros quatro bytes da chave expandida, que forma uma palavra, ficam na primeira coluna da matriz **w**.

A cifra consiste em N rodadas, e o número delas depende do comprimento da chave: 10 rodadas para uma chave de 16 bytes, 12 para uma chave de 24 bytes e 14 para uma chave de 32 bytes (Tabela 5.1). As primeiras $N - 1$ rodadas consistem em quatro funções de transformação distintas: SubBytes, ShiftRows, MixColumns e AddRoundKey, que serão descritas mais adiante. A rodada final contém apenas três transformações, e há uma transformação inicial única (AddRoundKey) antes da primeira rodada, o que pode ser considerado Rodada 0.

² No FIPS PUB 197, um número hexadecimal é indicado com delimitação de chaves. Usamos essa convenção neste capítulo.

³ No restante da discussão, as referências a $GF(2^8)$ são feitas ao corpo finito definido com esse polinômio.

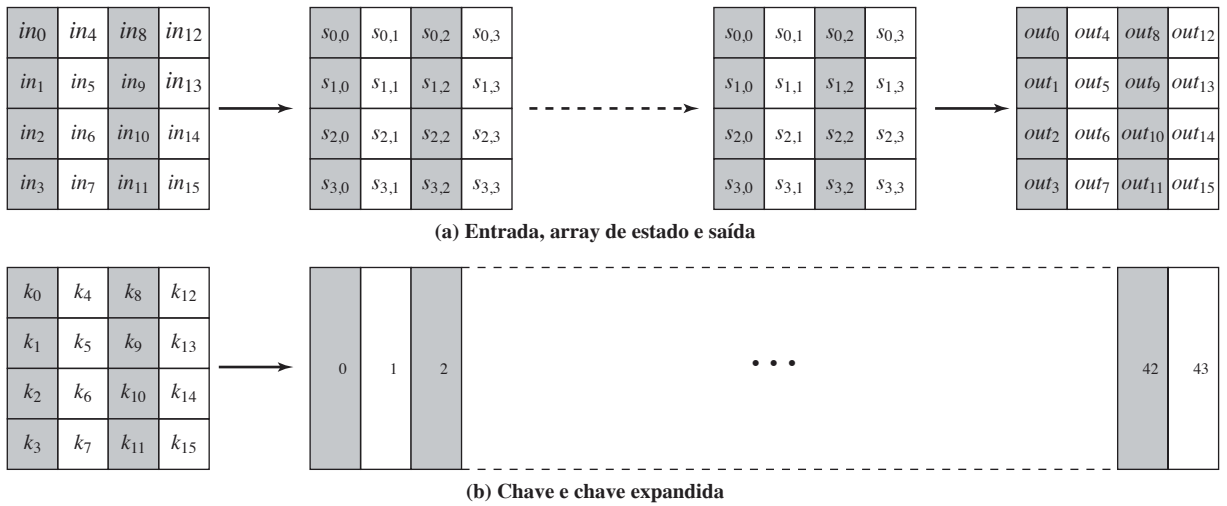
Figura 5.1 Processo de encriptação do AES.



Cada transformação usa uma ou mais matrizes de 4×4 como entrada e produz uma de 4×4 como saída. A Figura 5.1 mostra que a saída de cada rodada é uma matriz de 4×4 , com a saída da rodada final sendo o texto cifrado. Além disso, a função de expansão de chave gera $N + 1$ chaves de rodada, cada uma das quais é uma matriz distinta de 4×4 . Cada chave de rodada serve como uma das entradas para a transformação AddRoundKey em cada rodada.

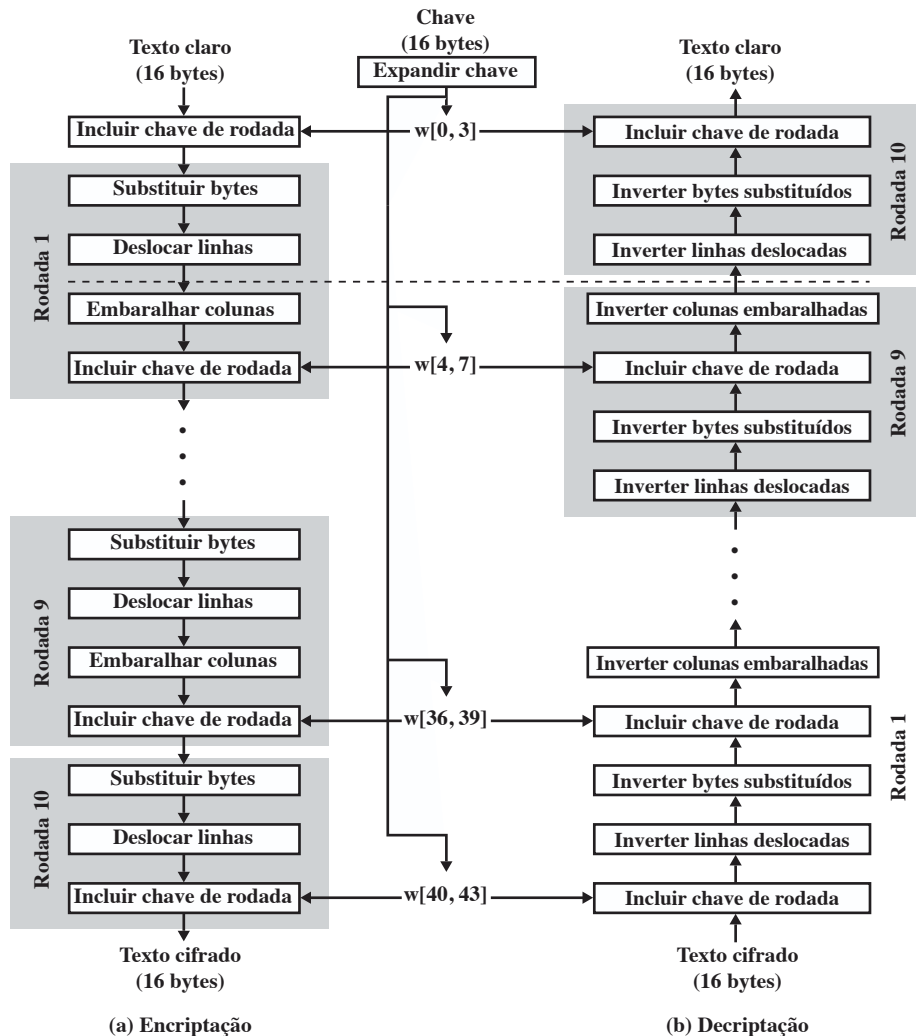
Tabela 5.1 Parâmetros do AES.

Tamanho da chave (words/bytes/bits)	4/16/128	6/24/192	8/32/256
Tamanho do bloco de texto claro (words/bytes/bits)	4/16/128	4/16/128	4/16/128
Número de rodadas	10	12	14
Tamanho da chave de rodada (words/bytes/bits)	4/16/128	4/16/128	4/16/128
Tamanho da chave expandida (words/bytes)	44/176	52/208	60/240

Figura 5.2 Estruturas de dados do AES.

Estrutura detalhada

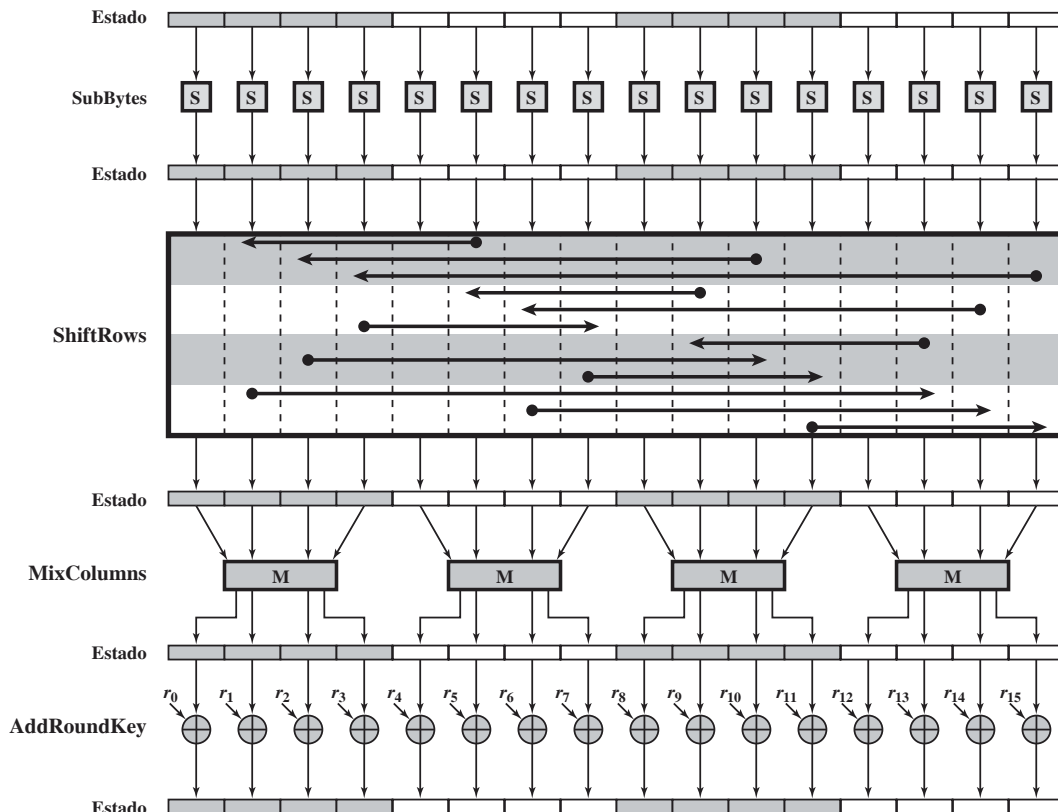
A Figura 5.3 mostra a cifra AES com mais detalhes, indicando a sequência de transformações em cada rodada e a função de deciptação correspondente. Como fizemos no Capítulo 3, apresentamos o procedimento de encriptação mais abaixo e o de deciptação no alto.

Figura 5.3 Encriptação e deciptação no AES.

Antes de mergulharmos nos detalhes, podemos fazer vários comentários sobre a estrutura geral do AES:

1. Um recurso digno de nota é que ela não é uma estrutura Feistel. Lembre-se de que, na estrutura Feistel clássica, metade do bloco de dados é usada para modificar a outra metade, e depois elas são invertidas. Em vez disso, AES processa o bloco de dados inteiro como uma única matriz durante cada rodada usando substituições e permutação.
2. A chave que é fornecida como entrada é expandida para um array de quarenta e quatro words de 32 bits cada um, $w[i]$. Quatro words distintas (128 bits) servem como uma chave para cada rodada; estas estão indicadas na Figura 5.3.
3. Quatro estágios diferentes são usados, um de permutação e três de substituição:
 - **SubBytes:** utiliza uma S-box para realizar uma substituição byte a byte do bloco
 - **ShiftRows:** uma permutação simples
 - **MixColumns:** uma substituição que utiliza aritmética sobre $GF(2^8)$
 - **AddRoundKey:** um XOR bit a bit simples do bloco atual com uma parte da chave expandida
4. A estrutura é muito simples. Para a encriptação e decriptação, a cifra começa com um estágio AddRoundKey, seguido por nove rodadas, e cada uma inclui todos os quatro estágios, seguidas por uma décima rodada de três estágios. A Figura 5.4 representa a estrutura de uma rodada de encriptação completa.
5. Somente o estágio AddRoundKey utiliza a chave. Por esse motivo, a cifra começa e termina com ele. Qualquer outro estágio, aplicado no início ou no fim, é reversível sem conhecimento da chave e, portanto, não impacta na segurança.
6. O estágio AddRoundKey, com efeito, é uma forma de cifra de Vernam, e por si só não seria formidável. Os outros três estágios oferecem confusão, difusão e não linearidade, mas isolados não ofereceriam segurança, pois não usam a chave. Podemos ver a cifra como operações alternadas de encriptação XOR

Figura 5.4 Rodada de encriptação do AES.



- (AddRoundKey) de um bloco, seguidas por embaralhamento deste (os outros três estágios), acompanhado por encriptação XOR, e assim por diante. Esse esquema é tanto eficiente quanto altamente seguro.
7. Cada estágio é facilmente reversível. Para os estágios SubByte, ShiftRows e MixColumns, uma função inversa é usada no algoritmo de deciptação. Para o AddRoundKey, o inverso é obtido pelo XOR da mesma chave da rodada com o bloco, usando o resultado de que $A \oplus B \oplus B = A$.
 8. Assim como na maioria das cifras em bloco, o algoritmo de deciptação utiliza a chave expandida em ordem reversa. Porém, o algoritmo de deciptação não é idêntico ao de encriptação. Isso é uma consequência da estrutura do AES em particular.
 9. Uma vez estabelecido que todos os quatro estágios são reversíveis, é fácil verificar que a deciptação recupera o texto claro. A Figura 5.3 descreve a encriptação e a deciptação indo em direções verticais opostas. Em cada ponto horizontal (por exemplo, a linha tracejada na figura), o **Estado** é o mesmo para encriptação e deciptação.
 10. A rodada final da encriptação e da deciptação consiste em apenas três estágios. Novamente, isso é uma consequência da estrutura do AES em particular, sendo necessário para tornar a cifra reversível.

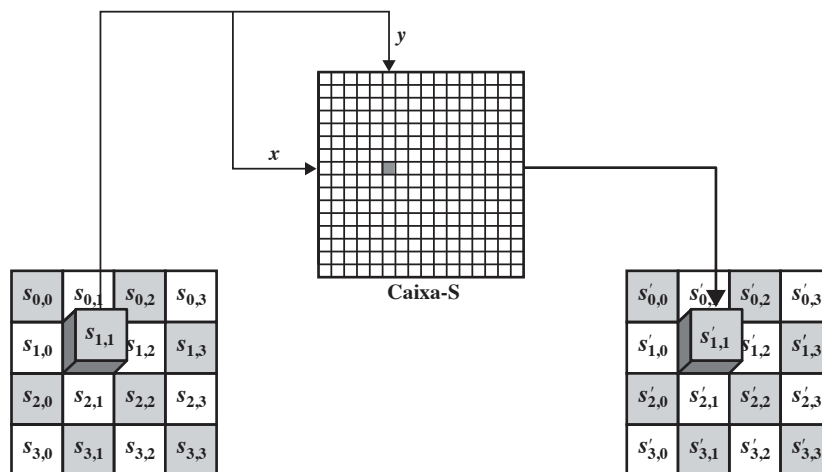
5.3 FUNÇÕES DE TRANSFORMAÇÃO DO AES

Agora, vamos passar para uma discussão sobre cada uma das quatro transformações usadas no AES. Para cada estágio, descrevemos o algoritmo em sentido direto (encriptação), o algoritmo em sentido inverso (decip-tação) e o raciocínio para tal estágio.

Transformação de SubBytes

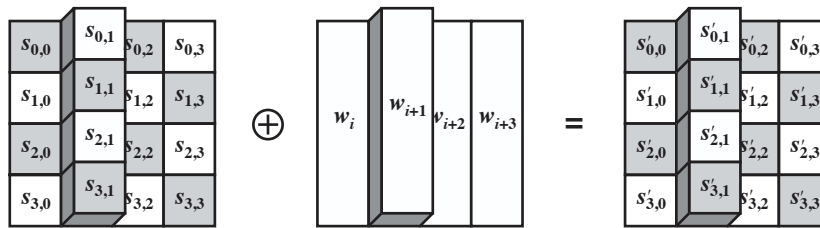
TRANSFORMAÇÕES DIRETA E INVERSA A **transformação direta de substituição de byte**, chamada SubBytes, consiste de uma simples pesquisa em tabela (Figura 5.5a). AES define uma matriz de 16×16 de valores de byte, chamada de S-box (Tabela 5.2a), que contém uma permutação de todos os 256 valores possíveis de 8 bits. Cada byte individual de **Estado** é mapeado para um novo byte da seguinte maneira: os 4 bits mais à esquerda do byte são usados como um valor de linha e os 4 bits mais à direita, como um valor de coluna. Esses valores de linha e coluna servem como índices para a S-box a fim de selecionar um valor de saída de 8 bits. Por exemplo, o valor hexadecimal {95} referencia a linha 9, coluna 5 da S-box, que contém o valor {2A}. De acordo com isso, o valor {95} é mapeado para o {2A}.

Figura 5.5 Operações em nível de byte do AES.



(a) Transformação de substituição de byte

(Continua)

Figura 5.5 Operações em nível de byte do AES. (continuação)**(b) Transformação de adição de chave de rodada****Tabela 5.2** S-box do AES.

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

(a) S-box

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

(b) S-box

Aqui está um exemplo da transformação SubBytes:

EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

→

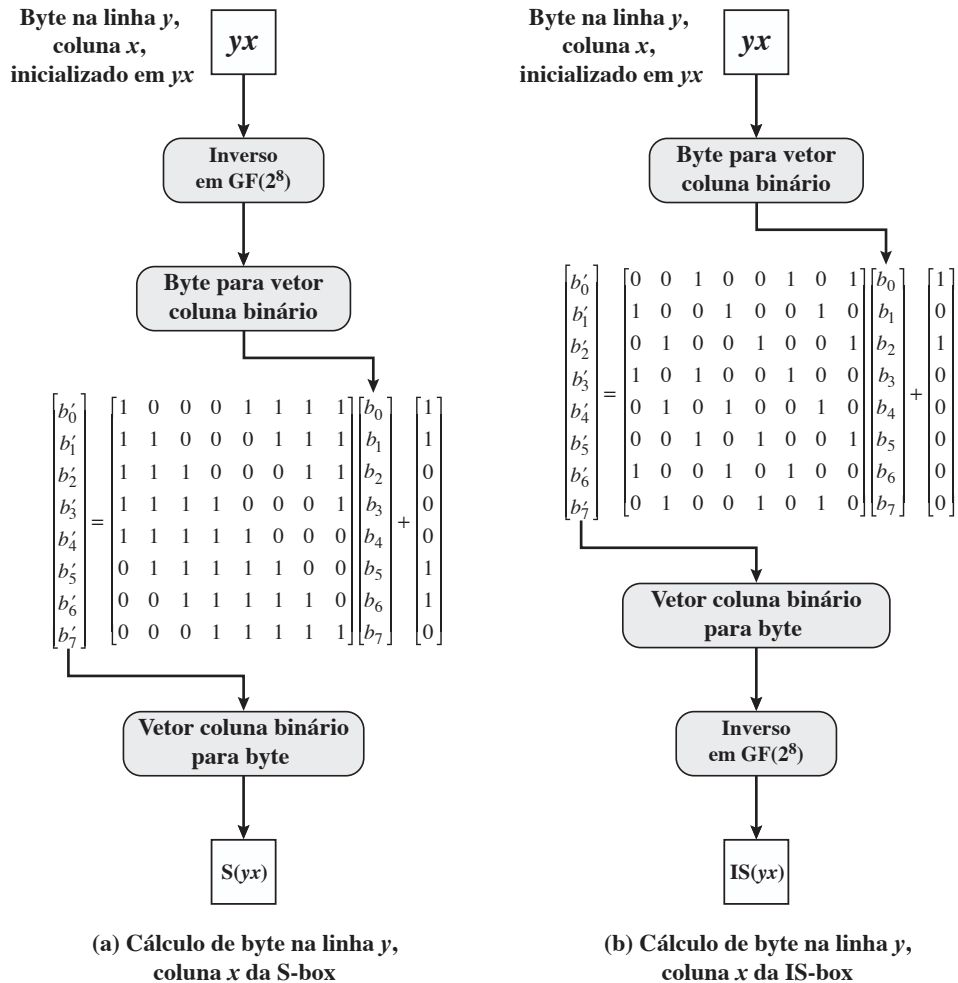
87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

A S-box é construída da seguinte forma (Figura 5.6a):

1. Inicialize a S-box com os valores em byte em sequência crescente linha por linha. A primeira linha contém {00}, {01}, {02}, ..., {0F}; a segunda linha contém {10}, {11} etc.; e assim por diante. Desse modo, o valor do byte na linha y , coluna x é $\{yx\}$.
2. Mapeie cada byte na S-box com seu inverso multiplicativo no corpo finito $\text{GF}(2^8)$; o valor {00} é mapeado consigo mesmo.
3. Considere que cada byte na S-box consiste em 8 bits rotulados $(b_7, b_6, b_5, b_4, b_3, b_2, b_1, b_0)$. Aplique a seguinte transformação a cada bit de cada byte na S-box:

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i \quad (5.1)$$

Figura 5.6 Construção da S-box e da IS-box.



onde c_i é o i -ésimo bit do byte c com o valor {63}; ou seja, $(c_7c_6c_5c_4c_3c_2c_1c_0) = (01100011)$. Aspas simples (') indicam que a variável deve ser atualizada pelo valor à direita. O padrão AES representa essa transformação em forma matricial da seguinte maneira:

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} \quad (5.2)$$

A Equação 5.2 precisa ser interpretada cuidadosamente. Na multiplicação de matriz comum,⁴ cada elemento na matriz produto é a soma dos produtos dos elementos de uma linha e uma coluna. Nesse caso, cada elemento na matriz produto é o XOR bit a bit de produtos de elementos de uma linha e uma coluna. Além disso, a adição final mostrada na Equação 5.2 é um XOR bit a bit. Relembre na Seção 4.7 que o XOR bit a bit é adicionado em $GF(2^8)$.

Como um exemplo, considere o valor de entrada {95}. O inverso multiplicativo em $GF(2^8)$ é $\{95\}^{-1} = \{8A\}$, que é 10001010 em binário. Usando a Equação 5.2,

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

O resultado é {2A}, que deverá aparecer na linha {09}, coluna {05} da S-box. Isso é verificado na Tabela 5.2a.

A **transformação invertida de substituição de byte**, chamada InvSubBytes, utiliza a S-box inversa mostrada na Tabela 5.2b. Observe, por exemplo, que a entrada {2A} produz a saída {95}, e a entrada {95} para a S-box gera {2A}. A S-box invertida é construída (Figura 5.6b) aplicando-se o inverso da transformação na Equação 5.1, seguido pelo inverso multiplicativo em $GF(2^8)$. A transformação inversa é:

$$b_i' = b_{(i+2) \bmod 8} \oplus b_{(i+5) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus d_i$$

onde byte $d = \{05\}$, ou 00000101. Podemos representar essa transformação da seguinte forma:

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

⁴ Para ter uma rápida visão geral das regras de multiplicação de matriz e vetor, consulte o Apêndice E (em <sv.pearson.com.br>, em inglês).

Para ver se InvSubBytes é o inverso de SubBytes, rotule as matrizes em SubBytes e InvSubBytes como $\mathbf{X}$ e $\mathbf{Y}$, respectivamente, e as versões vetoriais das constantes $\mathbf{c}$ e $\mathbf{d}$ como $\mathbf{C}$ e $\mathbf{D}$, respectivamente. Para algum vetor de 8 bits $\mathbf{B}$, a Equação 5.2 torna-se $\mathbf{B}' = \mathbf{XB} \oplus \mathbf{C}$. Precisamos mostrar que $\mathbf{Y}(\mathbf{XB} \oplus \mathbf{C}) \oplus \mathbf{D} = \mathbf{B}$. Multiplicando, temos que mostrar que $\mathbf{YXB} \oplus \mathbf{YC} \oplus \mathbf{D} = \mathbf{B}$. Isso se torna:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} \oplus$$

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \end{bmatrix}$$

Demonstramos que $\mathbf{YX}$ é igual à matriz identidade, e $\mathbf{YC} = \mathbf{D}$, de modo que $\mathbf{YC} \oplus \mathbf{D}$ corresponde ao vetor nulo.

RACIOCÍNIO A caixa-S é projetada para ser resistente a ataques criptoanalíticos conhecidos. Especificamente, os desenvolvedores do Rijndael buscaram um modelo que tivesse uma baixa correlação entre bits de entrada e bits de saída, e a propriedade de que a saída não pode ser descrita como uma função matemática simples da entrada [DAEM01]. A não linearidade é devida ao uso do inverso multiplicativo. Além disso, a constante na Equação 5.1 foi escolhida de modo que a S-box não tenha pontos fixos [$S\text{-box}(a) = a$] nem “pontos fixos opostos” [$S\text{-box}(a) = \bar{a}$], onde $\bar{a}$ é o complemento bit a bit de a .

Naturalmente, a S-box precisa ser reversível, ou seja, $IS\text{-box}[S\text{-box}(a)] = a$. Porém a S-box não é autoinversa no sentido de que não é verdadeiro que $S\text{-box}(a) = IS\text{-box}(a)$. Por exemplo, $S\text{-box}(\{95\}) = \{2A\}$, mas $IS\text{-box}(\{95\}) = \{AD\}$.

Transformação ShiftRows

TRANSFORMAÇÕES DIRETA E INVERSA A transformação direta de deslocamento de linhas, chamada ShiftRows, é representada na Figura 5.7a. A primeira linha de **Estado** não é alterada. Para a segunda linha, é realizado um deslocamento circular à esquerda por 1 byte. Para a terceira linha, é feito um deslocamento circular à esquerda por 2 bytes. Para a quarta linha, ocorre um deslocamento circular à esquerda por 3 bytes. A seguir vemos um exemplo de ShiftRows.

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

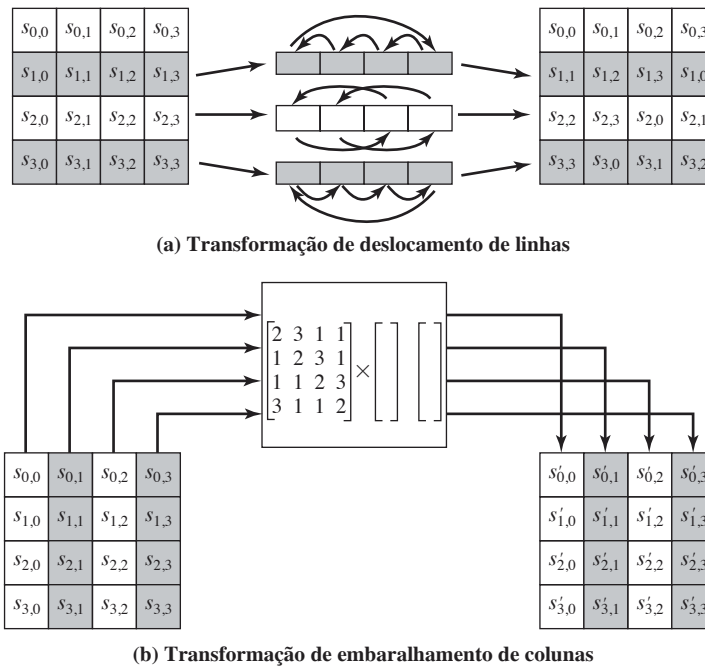
→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

A **transformação inversa de deslocamento de linhas**, chamada **InvShiftRows**, realiza os deslocamentos circulares na direção oposta para cada uma das três últimas linhas, com um deslocamento circular à direita por um byte para a segunda linha, e assim por diante.

RACIOCÍNIO A transformação de deslocamento de linhas é mais substancial do que pode parecer a princípio. Isso porque o **Estado**, além da entrada e saída da cifra, é tratado como um array de quatro colunas de 4 bytes. Assim, na encriptação, os quatro primeiros bytes do texto claro são copiados para a primeira coluna de **Estado**, e assim por diante. Além disso, conforme veremos, a chave da rodada é aplicada ao **Estado** coluna por coluna. Assim, um deslocamento de linha move um único byte de uma coluna para outra, que está a uma distância linear de um múltiplo de 4 bytes. Observe também que a transformação garante que os 4 bytes de uma coluna são espalhados para quatro colunas diferentes. A Figura 5.4 ilustra esse efeito.

Figura 5.7 Operações de linha e coluna do AES.



Transformação MixColumns

TRANSFORMAÇÕES DIRETA E INVERSA A **transformação direta de embaralhamento de colunas**, chamada **MixColumns**, opera sobre cada coluna individualmente. Cada byte de uma coluna é mapeado para um novo valor que é determinado em função de todos os quatro bytes nessa coluna. A transformação pode ser definida pela seguinte multiplicação de matriz sobre **Estado** (Figura 5.7b).

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix} \quad (5.3)$$

Cada elemento na matriz de produtos é a soma dos produtos dos elementos de uma linha e uma coluna. Nesse caso, as adições e multiplicações⁵ individuais são realizadas em $\text{GF}(2^8)$. A transformação MixColumns sobre uma única coluna de **Estado** pode ser expressa como

$$\begin{aligned} s_{0,j} &= (2 \cdot s_{0,j}) \oplus (3 \cdot s_{1,j}) \oplus s_{2,j} \oplus s_{3,j} \\ s_{1,j} &= s_{0,j} \oplus (2 \cdot s_{1,j}) \oplus (3 \cdot s_{2,j}) \oplus s_{3,j} \\ s_{2,j} &= s_{0,j} \oplus s_{1,j} \oplus (2 \cdot s_{2,j}) \oplus (3 \cdot s_{3,j}) \\ s_{3,j} &= (3 \cdot s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 \cdot s_{3,j}) \end{aligned} \quad (5.4)$$

A seguir vemos um exemplo de MixColumns:

87	F2	4D	97		47	40	A3	4C
6E	4C	90	EC		37	D4	70	9F
46	E7	4A	C3		94	E4	3A	42
A6	8C	D8	95		ED	A5	A6	BC

Vamos verificar a primeira coluna deste exemplo. Lembre-se, pela Seção 4.7, de que, em $\text{GF}(2^8)$, a adição é a operação XOR bit a bit, e que a multiplicação pode ser realizada de acordo com a regra estabelecida na Equação 4.14. Em particular, a multiplicação de um valor por x (ou seja, por {02}) pode ser implementada como um deslocamento à esquerda por 1 bit seguido de um XOR bit a bit com (0001 1011), caso o bit mais à esquerda do valor original (antes do deslocamento) seja 1. Assim, para verificar a transformação MixColumns na primeira coluna, precisamos mostrar que

$$\begin{aligned} (\{02\} \cdot \{87\}) \oplus (\{03\} \cdot \{6E\}) \oplus \{46\} \oplus \{A6\} &= \{47\} \\ \{87\} \oplus (\{02\} \cdot \{6E\}) \oplus (\{03\} \cdot \{46\}) \oplus \{A6\} &= \{37\} \\ \{87\} \oplus \{6E\} \oplus (\{02\} \cdot \{46\}) \oplus (\{03\} \cdot \{A6\}) &= \{94\} \\ (\{03\} \cdot \{87\}) \oplus \{6E\} \oplus \{46\} \oplus (\{02\} \cdot \{A6\}) &= \{ED\} \end{aligned}$$

Para a primeira equação, temos $\{02\} \cdot \{87\} = (0000 \ 1110) \oplus (0001 \ 1011) = (0001 \ 0101)$; e $\{03\} \cdot \{6E\} = \{6E\} \oplus (\{02\} \cdot \{6E\}) = (0110 \ 1110) \oplus (1101 \ 1100) = (1011 \ 0010)$. Então

$$\begin{aligned} \{02\} \cdot \{87\} &= 0001 \ 0101 \\ \{03\} \cdot \{6E\} &= 1011 \ 0010 \\ \{46\} &= 0100 \ 0110 \\ \{A6\} &= 1010 \ 0110 \\ &\underline{0100 \ 0111} = \{47\} \end{aligned}$$

As outras equações podem ser verificadas de modo semelhante.

A **transformação inversa de embaralhamento de colunas**, chamada InvMixColumns, é definida pela seguinte multiplicação de matrizes:

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \quad (5.5)$$

⁵ Seguimos a convenção do FIPS PUB 197 e usamos o símbolo $\cdot$ para indicar a multiplicação sobre o corpo finito $\text{GF}(2^8)$, e $\oplus$ para o XOR bit a bit, que corresponde à adição em $\text{GF}(2^8)$.

Fica imediatamente claro que a Equação 5.5 é o **inverso** da Equação 5.3. Precisamos mostrar que:

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

que é equivalente a mostrar que:

$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5.6)$$

Ou seja, a matriz de transformação inversa vezes a de transformação direta é igual à matriz identidade. Para verificar a primeira coluna da Equação 5.6, precisamos mostrar que:

$$\begin{aligned} (\{0E\} \{02\}) \oplus \{0B\} \oplus \{0D\} \oplus (\{09\} \{03\}) &= \{01\} \\ (\{09\} \{02\}) \oplus \{0E\} \oplus \{0B\} \oplus (\{0D\} \{03\}) &= \{00\} \\ (\{0D\} \{02\}) \oplus \{09\} \oplus \{0E\} \oplus (\{0B\} \{03\}) &= \{00\} \\ (\{0B\} \{02\}) \oplus \{0D\} \oplus \{09\} \oplus (\{0E\} \{03\}) &= \{00\} \end{aligned}$$

Para a primeira equação, temos $\{0E\} \cdot \{02\} = 00011100$ e $\{09\} \cdot \{03\} = \{09\} \oplus (\{09\} \cdot \{02\}) = 00001001 \oplus 00010010 = 00011011$. Então

$$\begin{aligned} \{0E\} \{02\} &= 00011100 \\ \{0B\} &= 00001011 \\ \{0D\} &= 00001101 \\ \{09\} \{03\} &= \underline{00011011} \\ &00000001 \end{aligned}$$

As outras equações podem ser verificadas de modo semelhante.

O documento do AES descreve outra maneira de caracterizar a transformação MixColumns, que é em termos da aritmética de polinômios. No padrão, MixColumns é definido considerando-se cada coluna de **Estado** como sendo um polinômio de quarto grau com coeficientes em $GF(2^8)$. Cada coluna é multiplicada módulo $(x^4 + 1)$ pelo polinômio fixo $a(x)$, dado por

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\} \quad (5.7)$$

O Apêndice 5A (<sv.pearson.com.br>, em inglês) demonstra que a multiplicação de cada coluna de **Estado** por $a(x)$ pode ser escrita como a multiplicação de matrizes da Equação 5.3. De modo semelhante, podemos ver que a transformação na Equação 5.5 corresponde a tratar cada coluna como um polinômio de quarto grau e multiplicar por $b(x)$, dado por

$$b(x) = \{0B\}x^3 + \{0D\}x^2 + \{09\}x + \{0E\} \quad (5.8)$$

É possível mostrar rapidamente que $b(x) = a^{-1}(x) \bmod (x^4 + 1)$.

RACIOCÍNIO Os coeficientes da matriz na Equação 5.3 são baseados em um código linear com máxima distância entre as palavras, o que garante um bom embaralhamento entre os bytes de cada coluna. A transformação de embaralhamento de colunas combinada com a de deslocamento de linhas garante que, após algumas rodadas, todos os bits da saída dependam de todos os bits da entrada. Veja uma discussão sobre isso em [DAEM99].

Além disso, a escolha dos coeficientes em MixColumns, que são todos $\{01\}$, $\{02\}$ ou $\{03\}$, foi influenciada por aspectos de implementação. Conforme discutimos, a multiplicação por esses coeficientes envolve no máximo um deslocamento e um XOR. Os coeficientes em InvMixColumns são mais fáceis de serem implementados. Porém a encriptação foi considerada mais importante do que a decríptação por dois motivos:

1. Para os modos de cifra CFB e OFB (figuras 6.5 e 6.6, descritas no Capítulo 6), somente a encriptação é usada.

2. Assim como qualquer cifra em bloco, AES pode ser usado para construir um código de autenticação de mensagem (Capítulo 12), e para isso apenas a encriptação é utilizada.

Transformação AddRoundKey

TRANSFORMAÇÕES DIRETA E INVERSA Na **transformação direta de adição de chave da rodada**, chamada AddRoundKey, os 128 bits de **Estado** passam por um XOR bit a bit com os 128 bits da chave da rodada. Como vemos na Figura 5.5b, a operação é vista como uma do tipo coluna por coluna entre os 4 bytes da coluna **Estado** e uma word da chave da rodada; ela também pode ser vista como uma operação em nível de byte. A seguir está um exemplo de AddRoundKey:

47	40	A3	4C		AC	19	28	57		EB	59	8B	1B
37	D4	70	9F		77	FA	D1	5C		40	2E	A1	C3
94	E4	3A	42		66	DC	29	00		F2	38	13	42
ED	A5	A6	BC		F3	21	41	6A		1E	84	E7	D6

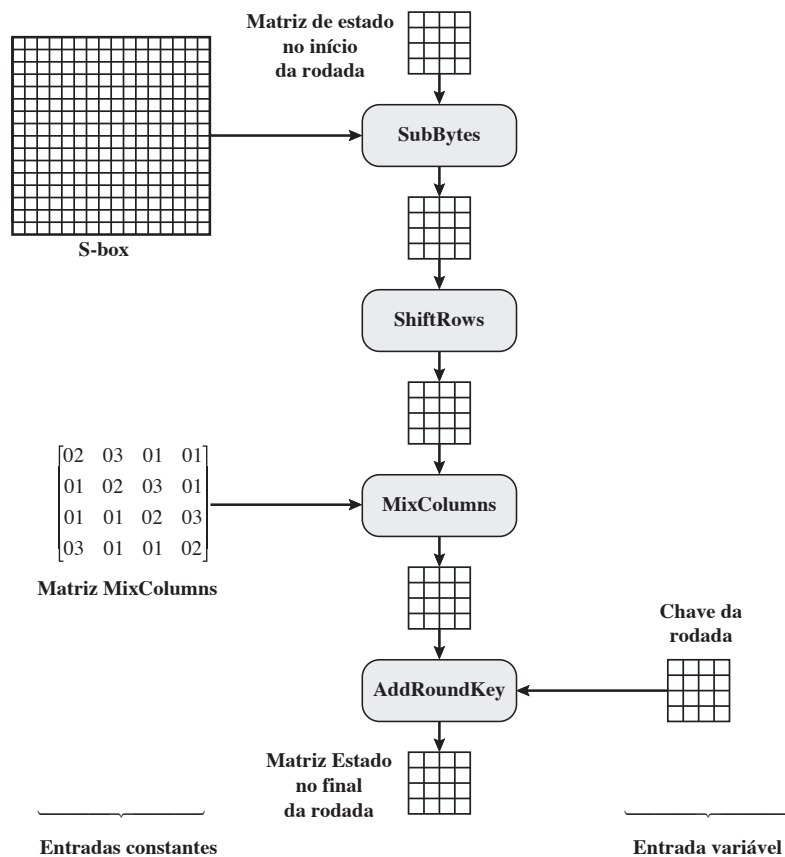
A primeira matriz é o **Estado**, e a segunda, a chave da rodada.

A **transformação inversa de adição de chave da rodada** é idêntica à direta de adição de chave da rodada, pois a operação XOR é o seu próprio inverso.

RACIOCÍNIO A transformação de adição de chave da rodada é a mais simples possível, e afeta cada bit de **Estado**. A complexidade da expansão de chave da rodada, mais a dos outros estágios do AES, garantem a sua segurança.

A Figura 5.8 apresenta outra visão de uma rodada única do AES, enfatizando os mecanismos e entradas de cada transformação.

Figura 5.8 Entradas para rodada única do AES.



5.4 EXPANSÃO DE CHAVE DO AES

Algoritmo de expansão de chave

O algoritmo de expansão de chave do AES utiliza como entrada uma chave de 4 words (16 bytes) e produz um array linear de 44 words (176 bytes). Isso é suficiente para oferecer uma chave da rodada de 4 words para o estágio AddRoundKey inicial e para cada uma das 10 rodadas da cifra. O pseudocódigo a seguir descreve a expansão:

```

KeyExpansion (byte key[16], word w[44])
{
    word temp
    for (i = 0; i < 4; i++)    w[i] = (key[4*i], key[4*i+1],
                                   key[4*i+2],
                                   key[4*i+3]);

    for (i = 4; i < 44; i++)
    {
        temp = w[i - 1];
        if (i mod 4 = 0)    temp = SubWord (RotWord (temp))
                                $\oplus$  Rcon[i/4];

        w[i] = w[i-4]  $\oplus$  temp
    }
}

```

A chave é copiada para as quatro primeiras words da chave expandida. O restante da chave expandida é preenchido com quatro words de cada vez. Cada word incluída $w[i]$ depende da imediatamente anterior, $w[i - 1]$, e da word quatro posições atrás, $w[i - 4]$. Em três dentre quatro casos, um simples XOR é usado. Para uma word cuja posição no array w é um múltiplo de 4, uma função mais complexa é empregada. A Figura 5.9 ilustra a geração das oito primeiras words da chave expandida, com o símbolo g para representar essa função complexa. A função g consiste nas seguintes subfunções:

1. RotWord realiza um deslocamento circular de um byte à esquerda em uma word. Isso significa que uma word de entrada $[B_0, B_1, B_2, B_3]$ é transformada em $[B_1, B_2, B_3, B_0]$.
2. SubWord realiza uma substituição byte a byte de sua word de entrada, usando a S-box (Tabela 5.2a).
3. O resultado das etapas 1 e 2 passa por um XOR com a constante da rodada, $Rcon[j]$.

A constante da rodada é uma word em que os três bytes mais à direita são sempre 0. Assim, o efeito de um XOR de uma word com Rcon se resume a realizar um XOR no byte mais à esquerda da word. A constante da rodada é diferente para cada uma delas, e é definida como $Rcon[j] = (RC[j], 0, 0, 0)$, com $RC[1] = 1$, $RC[j] = 2 \cdot RC[j - 1]$ e com a multiplicação definida sobre o corpo $GF(2^8)$. Os valores de $RC(j)$ em hexadecimal são

j	1	2	3	4	5	6	7	8	9	10
RC[j]	01	02	04	08	10	20	40	80	1B	36

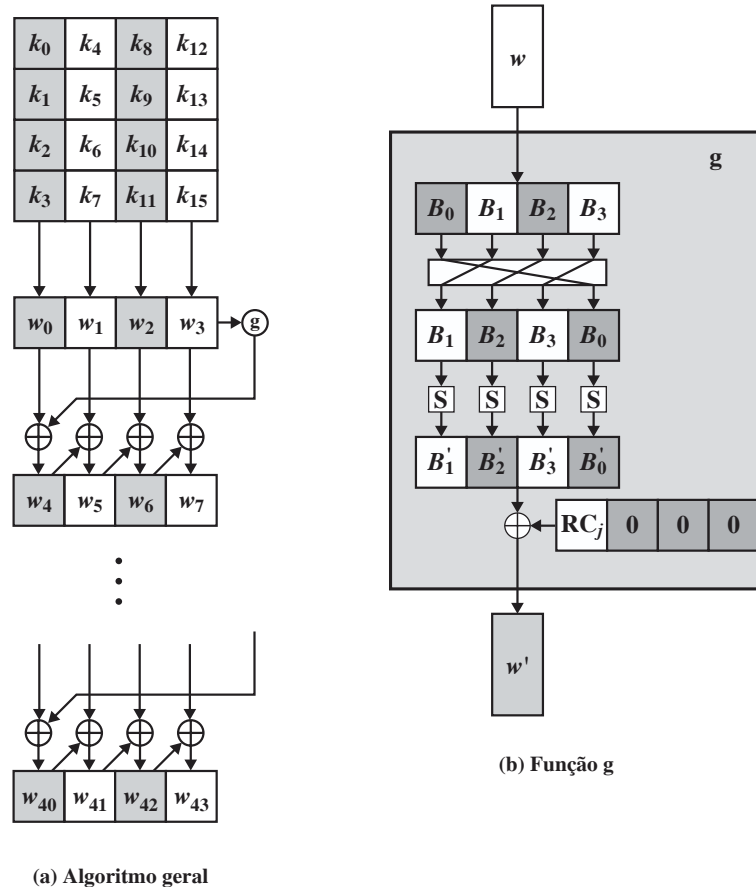
Por exemplo, suponha que a chave da rodada 8 seja

EA D2 73 21 B5 8D BA D2 31 2B F5 60 7F 8D 29 2F

Então, os 4 primeiros bytes (primeira coluna) da chave da rodada 9 são calculados da seguinte forma:

i (decimal)	temp	Após RotWord	Após SubWord	Rcon (9)	Após XOR com Rcon	$w[i-4]$	$w[i] = \text{temp} \oplus w[i-4]$
36	7F8D292F	8D292F7F	5DA515D2	1B000000	46A515D2	EAD27321	AC7766F3

Figura 5.9 Expansão de chave do AES.



Raciocínio

Os desenvolvedores do Rijndael criaram o algoritmo de expansão de chave para ser resistente a ataques criptoanalíticos conhecidos. A inclusão de uma constante dependente da rodada elimina a simetria, ou similaridade, entre as formas como as chaves da rodada são geradas em diferentes rodadas. Os critérios específicos que foram usados são os seguintes [DAEM99]:

- O conhecimento de uma parte da chave da cifra ou chave da rodada não permite o cálculo de muitos outros bits dela.
- Uma transformação reversível [ou seja, o conhecimento de quaisquer Nk words consecutivas da chave expandida permite a regeneração da chave expandida inteira (Nk = tamanho da chave em words)].
- Velocidade em uma grande gama de processadores.
- Uso de constantes de rodada para eliminar simetrias.
- Difusão de diferenças de chave de cifra nas chaves da rodada; ou seja, cada bit de chave afeta muitos bits de chave da rodada.
- Não linearidade suficiente para proibir a determinação total das diferenças de chave da rodada somente a partir das diferenças da chave de cifra.
- Simplicidade de descrição.

Os autores não quantificam o primeiro ponto da lista anterior, mas a ideia é que, se você souber menos do que Nk words consecutivas da chave de cifra ou de uma das chaves da rodada, então será difícil reconstruir os bits desconhecidos restantes. Quanto menos bits alguém souber, mais difícil é realizar a reconstrução ou determinar outros bits na expansão da chave.

5.5 EXEMPLO DE AES

Vamos agora trabalhar um exemplo e considerar algumas de suas implicações. Embora não se espere do leitor que simule o exemplo à mão, será informativo estudar os padrões hexadecimais que ocorrem de uma etapa para a outra.

Para este exemplo, o texto claro é um palíndromo hexadecimal. O texto claro, a chave e o texto cifrado resultante são

Texto claro:	0123456789abcdeffedcba9876543210
Chave:	0f1571c947d9e8590cb7add6af7f6798
Texto cifrado:	ff0b844a0853bf7c6934ab4364148fb9

Resultados

A Tabela 5.3 mostra a expansão da chave de 16 bytes para 10 chaves de rodada. Como explicado anteriormente, esse processo é realizado word a word, com cada uma de quatro bytes ocupando uma coluna da matriz de chave de rodada. A coluna da esquerda mostra as quatro words de chave de rodada geradas para cada rodada. Já a coluna da direita mostra os passos usados para gerar a word auxiliar empregada na expansão da chave. Começamos, é claro, com a chave em si servindo como uma de rodada para a rodada 0.

Tabela 5.3 Expansão de chave para o exemplo de AES.

Words de chave	Função auxiliar
w0 = 0f 15 71 c9 w1 = 47 d9 e8 59 w2 = 0c b7 ad d6 w3 = af 7f 67 98	RotWord (w3) = 7f 67 98 af = x1 SubWord (x1) = d2 85 46 79 = y1 Rcon (1) = 01 00 00 00 y1 ⊕ Rcon (1) = d3 85 46 79 = z1
w4 = w0 ⊕ z1 = dc 90 37 b0 w5 = w4 ⊕ w1 = 9b 49 df e9 w6 = w5 ⊕ w2 = 97 fe 72 3f w7 = w6 ⊕ w3 = 38 81 15 a7	RotWord (w7) = 81 15 a7 38 = x2 SubWord (x2) = 0c 59 5c 07 = y2 Rcon (2) = 02 00 00 00 y2 ⊕ Rcon (2) = 0e 59 5c 07 = z2
w8 = w4 ⊕ z2 = d2 c9 6b b7 w9 = w8 ⊕ w5 = 49 80 b4 5e w10 = w9 ⊕ w6 = de 7e c6 61 w11 = w10 ⊕ w7 = e6 ff d3 c6	RotWord (w11) = ff d3 c6 e6 = x3 SubWord (x3) = 16 66 b4 83 = y3 Rcon (3) = 04 00 00 00 y3 ⊕ Rcon (3) = 12 66 b4 8e = z3
w12 = w8 ⊕ z3 = c0 af df 39 w13 = w12 ⊕ w9 = 89 2f 6b 67 w14 = w13 ⊕ w10 = 57 51 ad 06 w15 = w14 ⊕ w11 = b1 ae 7e c0	RotWord (w15) = ae 7e c0 b1 = x4 SubWord (x4) = e4 f3 ba c8 = y4 Rcon (4) = 08 00 00 00 y4 ⊕ Rcon (4) = ec f3 ba c8 = z4
w16 = w12 ⊕ z4 = 2c 5c 65 f1 w17 = w16 ⊕ w13 = a5 73 0e 96 w18 = w17 ⊕ w14 = f2 22 a3 90 w19 = w18 ⊕ w15 = 43 8c dd 50	RotWord (w19) = 8c dd 50 43 = x5 SubWord (x5) = 64 c1 53 1a = y5 Rcon (5) = 10 00 00 00 y5 ⊕ Rcon (5) = 74 c1 53 1a = z5
w20 = w16 ⊕ z5 = 58 9d 36 eb w21 = w20 ⊕ w17 = fd ee 38 7d w22 = w21 ⊕ w18 = 0f cc 9b ed w23 = w22 ⊕ w19 = 4c 40 46 bd	RotWord (w23) = 40 46 bd 4c = x6 SubWord (x6) = 09 5a 7a 29 = y6 Rcon (6) = 20 00 00 00 y6 ⊕ Rcon (6) = 29 5a 7a 29 = z6
w24 = w20 ⊕ z6 = 71 c7 4c c2 w25 = w24 ⊕ w21 = 8c 29 74 bf w26 = w25 ⊕ w22 = 83 e5 ef 52 w27 = w26 ⊕ w23 = cf a5 a9 ef	RotWord (w27) = a5 a9 ef cf = x7 SubWord (x7) = 06 d3 bf 8a = y7 Rcon (7) = 40 00 00 00 y7 ⊕ Rcon (7) = 46 d3 df 8a = z7

(continua)

(continuação)

Words de chave	Função auxiliar
$w28 = w24 \oplus z7 = 37\ 14\ 93\ 48$ $w29 = w28 \oplus w25 = bb\ 3d\ e7\ f7$ $w30 = w29 \oplus w26 = 38\ d8\ 08\ a5$ $w31 = w30 \oplus w27 = f7\ 7d\ a1\ 4a$	$RotWord(w31) = 7d\ a1\ 4a\ f7 = x8$ $SubWord(x8) = ff\ 32\ d6\ 68 = y8$ $Rcon(8) = 80\ 00\ 00\ 00$ $y8 \oplus Rcon(8) = 7f\ 32\ d6\ 68 = z8$
$w32 = w28 \oplus z8 = 48\ 26\ 45\ 20$ $w33 = w32 \oplus w29 = f3\ 1b\ a2\ d7$ $w34 = w33 \oplus w30 = cb\ c3\ aa\ 72$ $w35 = w34 \oplus w32 = 3c\ be\ 0b\ 3$	$RotWord(w35) = be\ 0b\ 38\ 3c = x9$ $SubWord(x9) = ae\ 2b\ 07\ eb = y9$ $Rcon(9) = 1b\ 00\ 00\ 00$ $y9 \oplus Rcon(9) = b5\ 2b\ 07\ eb = z9$
$w36 = w32 \oplus z9 = fd\ 0d\ 42\ cb$ $w37 = w36 \oplus w33 = 0e\ 16\ e0\ 1c$ $w38 = w37 \oplus w34 = c5\ d5\ 4a\ 6e$ $w39 = w38 \oplus w35 = f9\ 6b\ 41\ 56$	$RotWord(w39) = 6b\ 41\ 56\ f9 = x10$ $SubWord(x10) = 7f\ 83\ b1\ 99 = y10$ $Rcon(10) = 36\ 00\ 00\ 00$ $y10 \oplus Rcon(10) = 49\ 83\ b1\ 99 = z10$
$w40 = w36 \oplus z10 = b4\ 8e\ f3\ 52$ $w41 = w40 \oplus w37 = ba\ 98\ 13\ 4e$ $w42 = w41 \oplus w38 = 7f\ 4d\ 59\ 20$ $w43 = w42 \oplus w39 = 86\ 26\ 18\ 76$	

Em seguida, a Tabela 5.4 mostra a progressão de **Estado** através do processo de encriptação do AES. A primeira coluna indica o valor de **Estado** no início de uma rodada. Para a primeira linha, **Estado** é apenas a disposição em forma matricial do texto claro. A segunda, a terceira e a quarta colunas apresentam o valor de **Estado** para esta rodada após as transformações SubBytes, ShiftRows e MixColumns, respectivamente. A quinta coluna mostra a chave de rodada. Você pode verificar que essas chaves de rodada se equiparam com aquelas na Tabela 5.3. A primeira coluna exibe o valor de **Estado** resultante do XOR bit a bit de **Estado** após os MixColumns posteriores com a chave de rodada para a anterior.

Tabela 5.4 Exemplo do AES.

Início da rodada	Após SubBytes	Após ShiftRows	Após MixColumns	Chave de rodada
01 89 fe 76 23 ab dc 54 45 cd ba 32 67 ef 98 10				0f 47 0c af 15 d9 b7 7f 71 e8 ad 67 c9 59 d6 98
0e ce f2 d9 36 72 6b 2b 34 25 17 55 ae b6 4e 88	ab 8b 89 35 05 40 7f f1 18 3f f0 fc e4 4e 2f c4	ab 8b 89 35 40 7f f1 05 f0 fc 18 3f c4 e4 4e 2f	b9 94 57 75 e4 8e 16 51 47 20 9a 3f c5 d6 f5 3b	dc 9b 97 38 90 49 fe 81 37 df 72 15 b0 e9 3f a7
65 0f c0 4d 74 c7 e8 d0 70 ff e8 2a 75 3f ca 9c	4d 76 ba e3 92 c6 9b 70 51 16 9b e5 9d 75 74 de	4d 76 ba e3 c6 9b 70 92 9b e5 51 16 de 9d 75 74	8e 22 db 12 b2 f2 dc 92 df 80 f7 c1 2d c5 1e 52	d2 49 de e6 c9 80 7e ff 6b b4 c6 d3 b7 5e 61 c6
5c 6b 05 f4 7b 72 a2 6d b4 34 31 12 9a 9b 7f 94	4a 7f 6b bf 21 40 3a 3c 8d 18 c7 c9 b8 14 d2 22	4a 7f 6b bf 40 3a 3c 21 c7 c9 8d 18 22 b8 14 d2	b1 c1 0b cc ba f3 8b 07 f9 1f 6a c3 1d 19 24 5c	c0 89 57 b1 af 2f 51 ae df 6b ad 7e 39 67 06 c0
71 48 5c 7d 15 dc da a9 26 74 c7 bd 24 7e 22 9c	a3 52 4a ff 59 86 57 d3 f7 92 c6 7a 36 f3 93 de	a3 52 4a ff 86 57 d3 59 c6 7a f7 92 de 36 f3 93	d4 11 fe 0f 3b 44 06 73 cb ab 62 37 19 b7 07 ec	2c a5 f2 43 5c 73 22 8c 65 0e a3 dd f1 96 90 50

(continua)

(continuação)

Início da rodada	Após SubBytes	Após ShiftRows	Após MixColumns	Chave de rodada
f8 b4 0c 4c 67 37 24 ff ae a5 c1 ea e8 21 97 bc	41 8d fe 29 85 9a 36 16 e4 06 78 87 9b fd 88 65	41 8d fe 29 9a 36 16 85 78 87 e4 06 65 9b fd 88	2a 47 c4 48 83 e8 18 ba 84 18 27 23 eb 10 0a f3	58 fd 0f 4c 9d ee cc 40 36 38 9b 46 eb 7d ed bd
72 ba cb 04 1e 06 d4 fa b2 20 bc 65 00 6d e7 4e	40 f4 1f f2 72 6f 48 2d 37 b7 65 4d 63 3c 94 2f	40 f4 1f f2 6f 48 2d 72 65 4d 37 b7 2f 63 3c 94	7b 05 42 4a 1e d0 20 40 94 83 18 52 94 c4 43 fb	71 8c 83 cf c7 29 e5 a5 4c 74 ef a9 c2 bf 52 ef
0a 89 c1 85 d9 f9 c5 e5 d8 f7 f7 fb 56 7b 11 14	67 a7 78 97 35 99 a6 d9 61 68 68 0f b1 21 82 fa	67 a7 78 97 99 a6 d9 35 68 0f 61 68 fa b1 21 82	ec 1a c0 80 0c 50 53 c7 3b d7 00 ef b7 22 72 e0	37 bb 38 f7 14 3d d8 7d 93 e7 08 a1 48 f7 a5 4a
db a1 f8 77 18 6d 8b ba a8 30 08 4e ff d5 d7 aa	b9 32 41 f5 ad 3c 3d f4 c2 04 30 2f 16 03 0e ac	b9 32 41 f5 3c 3d f4 ad 30 2f c2 04 ac 16 03 0e	b1 1a 44 17 3d 2f ec b6 0a 6b 2f 42 9f 68 f3 b1	48 f3 cb 3c 26 1b c3 be 45 a2 aa 0b 20 d7 72 38
f9 e9 8f 2b 1b 34 2f 08 4f c9 85 49 bf bf 81 89	99 1e 73 f1 af 18 15 30 84 dd 97 3b 08 08 0c a7	99 1e 73 f1 18 15 30 af 97 3b 84 dd a7 08 08 0c	31 30 3a c2 ac 71 8c c4 46 65 48 eb 6a 1c 31 62	fd 0e c5 f9 0d 16 d5 6b 42 e0 4a 41 cb 1c 6e 56
cc 3e ff 3b a1 67 59 af 04 85 02 aa a1 00 5f 34	4b b2 16 e2 32 85 cb 79 f2 97 77 ac 32 63 cf 18	4b b2 16 e2 85 cb 79 32 77 ac f2 97 18 32 63 cf	4b 86 8a 36 b1 cb 27 5a fb f2 f2 af cc 5a 5b cf	b4 ba 7f 86 8e 98 4d 26 f3 13 59 18 52 4e 20 76
ff 08 69 64 0b 53 34 14 84 bf ab 8f 4a 7c 43 b9				

Efeito avalanche

Caso uma pequena alteração na chave ou no texto claro produzisse uma pequena mudança no texto cifrado correspondente, isso poderia ser usado para reduzir de forma significativa o tamanho do espaço de textos claros (ou chaves) possíveis a ser pesquisado. O que é desejado é o efeito avalanche, em que uma pequena mudança no texto claro ou na chave produz uma grande alteração no texto cifrado.

Usando o exemplo da Tabela 5.4, a Tabela 5.5 mostra o resultado quando o oitavo bit do texto claro é modificado. A segunda coluna da tabela exibe o valor da matriz **Estado** no final de cada rodada para os dois textos claros. Observe que, depois de apenas uma rodada, 20 bits do vetor **Estado** diferem. Após duas rodadas, cerca de metade dos pedaços diferem. Essa magnitude de diferença se propaga através de rodadas restantes. Uma distinção de bit em cerca de metade das posições é o resultado mais desejável. De maneira clara, se quase todos os bits forem alterados, isso seria logicamente equivalente a quase nenhum dos bits sendo alterados. Em outras palavras, ao selecionar dois textos claros ao acaso, espera-se que eles difiram em cerca de metade das posições de bits e os dois textos cifrados também se diferenciem em mais ou menos metade das posições.

A Tabela 5.6 indica a mudança dos valores da matriz **Estado** quando o texto claro é usado e as duas chaves diferem no oitavo bit. Isto é, para o segundo caso, a chave é 0e1571c947d9e8590cb7add6af7f6798. Mais uma vez, uma rodada produz uma mudança significativa, e a magnitude de alteração após todas as rodadas subseqüentes é cerca de metade dos bits. Assim, com base neste exemplo, o AES exibe um efeito avalanche muito forte.

Observe que esse efeito avalanche é mais forte do que aquele para o DES (Tabela 3.2), que requer três rodadas para chegar a um ponto em que cerca de metade dos bits é alterada, tanto para uma mudança de bit no texto claro quanto uma de bit na chave.

Tabela 5.5 Efeito avalanche no AES: mudança no texto claro.

Rodada		Número de bits que diferem
	0123456789abcdef fedcba9876543210 0023456789abcdef fedcba9876543210	1
0	0e3634aece7225b6f26b174ed92b5588 0f3634aece7225b6f26b174ed92b5588	1
1	657470750fc7ff3fc0e8e8ca4dd02a9c c4a9ad090fc7ff3fc0e8e8ca4dd02a9c	20
2	5c7bb49a6b72349b05a2317ff46d1294 fe2ae569f7ee8bb8c1f5a2bb37ef53d5	58
3	7115262448dc747e5cdac7227da9bd9c ec093dfb7c45343d689017507d485e62	59
4	f867aee8b437a5210c24c1974cffeabc 43efdb697244df808e8d9364ee0ae6f5	61
5	721eb200ba06206dcbd4bce704fa654e 7b28a5d5ed643287e006c099bb375302	68
6	0ad9d85689f9f77bc1c5f71185e5fb14 3bc2d8b6798d8ac4fe36a1d891ac181a	64
7	db18a8ffa16d30d5f88b08d777ba4eaa 9fb8b5452023c70280e5c4bb9e555a4b	67
8	f91b4fbfe934c9bf8f2f85812b084989 20264e1126b219aef7feb3f9b2d6de40	65
9	cca104a13e678500ff59025f3bafaa34 b56a0341b2290ba7dfdfbddcd8578205	61
10	ff0b844a0853bf7c6934ab4364148fb9 612b89398d0600cde116227ce72433f0	58

Tabela 5.6 Efeito avalanche no AES: mudança na chave.

Rodada		Número de bits que diferem
	0123456789abcdef fedcba9876543210 0123456789abcdef fedcba9876543210	0
0	0e3634aece7225b6f26b174ed92b5588 0f3634aece7225b6f26b174ed92b5588	1
1	657470750fc7ff3fc0e8e8ca4dd02a9c c5a9ad090ec7ff3fc1e8e8ca4cd02a9c	22
2	5c7bb49a6b72349b05a2317ff46d1294 90905fa9563356d15f3760f3b8259985	58
3	7115262448dc747e5cdac7227da9bd9c 18aeb7aa794b3b66629448d575c7cebf	67
4	f867aee8b437a5210c24c1974cffeabc f81015f993c978a876ae017cb49e7eec	63
5	721eb200ba06206dcbd4bce704fa654e 5955c91b4e769f3cb4a94768e98d5267	81
6	0ad9d85689f9f77bc1c5f71185e5fb14 dc60a24d137662181e45b8d3726b2920	70
7	db18a8ffa16d30d5f88b08d777ba4eaa fe8343b8f88bef66cab7e977d005a03c	74
8	f91b4fbfe934c9bf8f2f85812b084989 da7dad581d1725c5b72fa0f9d9d1366a	67
9	cca104a13e678500ff59025f3bafaa34 0ccb4c66bbfd912f4b511d72996345e0	59
10	ff0b844a0853bf7c6934ab4364148fb9 fc8923ee501a7d207ab670686839996b	53

5.6 IMPLEMENTAÇÃO DO AES

Cifra inversa equivalente

Conforme dissemos, a cifra de deciptação do AES não é idêntica à de encriptação (Figura 5.3). Ou seja, a sequência de transformações para a deciptação difere daquela para a encriptação, embora a forma dos escalonamentos de chave para encriptação e deciptação seja a mesma. Isso tem a desvantagem de que dois módulos de software e firmware separados são necessários para aplicações que exigem tanto encriptação quanto deciptação. Contudo, existe uma versão equivalente do algoritmo de deciptação que tem a mesma estrutura do algoritmo de encriptação. A versão equivalente tem a mesma sequência de transformações do algoritmo de encriptação (com transformações substituídas por seus inversos). Para conseguir essa equivalência, é preciso haver uma mudança no escalonamento de chave.

Duas mudanças separadas são necessárias para alinhar a estrutura de deciptação com a de encriptação. Como ilustrado na Figura 5.3, uma rodada de encriptação tem a estrutura SubBytes, ShiftRows, MixColumns, AddRoundKey. A rodada de deciptação padrão tem a estrutura InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns. Assim, os dois primeiros estágios da rodada de deciptação precisam ser trocados, e os dois seguintes também precisam ser trocados.

TROCANDO INVSHIFTROWS E INVSUBBYTES InvShiftRows afeta a sequência de bytes em **Estado**, mas não altera o conteúdo deles e não depende desse conteúdo para realizar sua transformação. InvSubBytes atinge o conteúdo dos bytes em **Estado**, mas não altera a sequência deles e não depende dela para realizar sua transformação. Assim, essas duas operações comutam e podem ser trocadas. Para determinado **Estado** S_i ,

$$\text{InvShiftRows} [\text{InvSubBytes} (S_i)] = \text{InvSubBytes} [\text{InvShiftRows} (S_i)]$$

TROCANDO ADDROUNDKEY E INV MIXCOLUMNS As transformações AddRoundKey e InvMixColumns não alteram a sequência de bytes em **Estado**. Se examinarmos a chave como uma sequência de words, então AddRoundKey e InvMixColumns operam sobre **Estado** uma coluna de cada vez. Essas duas operações são lineares com relação à entrada da coluna. Ou seja, para determinado **Estado** S_i e determinada chave de rodada w_j ,

$$\text{InvMixColumns} (S_i \oplus w_j) = [\text{InvMixColumns} (S_i)] \oplus [\text{InvMixColumns} (w_j)]$$

Para ver isso, suponha que a primeira coluna do **Estado** S_i seja a sequência (y_0, y_1, y_2, y_3) e que a primeira coluna de chave de rodada w_j seja (k_0, k_1, k_2, k_3) . Então, precisamos mostrar que

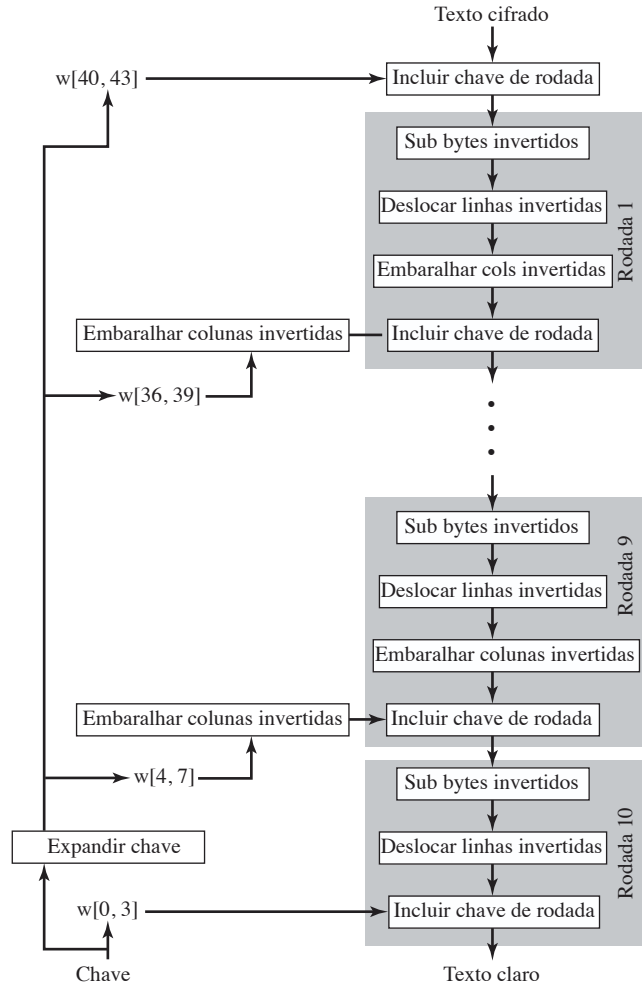
$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} y_0 \oplus k_0 \\ y_1 \oplus k_1 \\ y_2 \oplus k_2 \\ y_3 \oplus k_3 \end{bmatrix} = \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix} \oplus \begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} k_0 \\ k_1 \\ k_2 \\ k_3 \end{bmatrix}$$

Vamos demonstrar isso para a entrada da primeira coluna. É necessário indicar que:

$$\begin{aligned} & [[\{0E\} \ (y_0 \oplus k_0)] \oplus [\{0B\} \ (y_1 \oplus k_1)] \oplus [\{0D\} \ (y_2 \oplus k_2)] \oplus [\{09\} \ (y_3 \oplus k_3)]] \\ &= [[\{0E\} \ y_0] \oplus [\{0B\} \ y_1] \oplus [\{0D\} \ y_2] \oplus [\{09\} \ y_3]] \oplus \\ & \quad [[\{0E\} \ k_0] \oplus [\{0B\} \ k_1] \oplus [\{0D\} \ k_2] \oplus [\{09\} \ k_3]] \end{aligned}$$

Essa equação é válida por inspeção. Assim, podemos trocar AddRoundKey e InvMixColumns, desde que primeiro apliquemos InvMixColumns à chave da rodada. Observe que não precisamos empregar InvMixColumns à chave da rodada para a entrada da primeira transformação AddRoundKey (precedendo a primeira rodada), nem da última transformação AddRoundKey (na rodada 10). Isso porque essas duas transformações AddRoundKey não são trocadas com InvMixColumns para produzir o algoritmo de deciptação equivalente.

A Figura 5.10 ilustra o algoritmo de deciptação equivalente.

Figura 5.10 Cifra inversa equivalente.

Aspectos de implementação

A proposta Rijndael [DAEM99] oferece algumas sugestões para a implementação eficiente em processadores de 8 bits, típicos para os smart cards atuais, e em processadores de 32 bits, típica para PCs.

PROCESSADOR DE 8 BITS AES pode ser implementado de modo muito eficiente em um processador de 8 bits. AddRoundKey é uma operação XOR byte a byte. Já ShiftRows é uma operação simples de deslocamento de bytes. SubBytes opera em nível de byte e só exige uma tabela de 256 bytes.

A transformação MixColumns exige multiplicação de matriz no corpo $GF(2^8)$, o que significa que todas as operações são executadas sobre bytes. MixColumns só exige multiplicação por $\{02\}$ e $\{03\}$, que, como vimos, envolve deslocamentos simples, XORs condicionais e XORs. Isso pode ser implementado de uma maneira mais eficiente, que elimina os deslocamentos e XORs condicionais. O conjunto de equações 5.4 mostra as fórmulas para a transformação MixColumns em uma única coluna. Usando a identidade $\{03\} \cdot x = (\{02\} \cdot x) \oplus x$, podemos reescrever o conjunto de equações 5.4 da seguinte forma:

$$\begin{aligned}
 Tmp &= s_{0,j} \oplus s_{1,j} \oplus s_{2,j} \oplus s_{3,j} \\
 s_{0,j} &= s_{0,j} \oplus Tmp \oplus [2 \ (s_{0,j} \oplus s_{1,j})] \\
 s_{1,j} &= s_{1,j} \oplus Tmp \oplus [2 \ (s_{1,j} \oplus s_{2,j})] \\
 s_{2,j} &= s_{2,j} \oplus Tmp \oplus [2 \ (s_{2,j} \oplus s_{3,j})] \\
 s_{3,j} &= s_{3,j} \oplus Tmp \oplus [2 \ (s_{3,j} \oplus s_{0,j})]
 \end{aligned} \tag{5.9}$$

O conjunto de equações 5.9 é verificado pela expansão e eliminação de termos.

A multiplicação por {02} envolve um deslocamento e um XOR condicional. Essa implementação pode ser vulnerável a um ataque de temporização do tipo descrito na Seção 3.4. Para agir contra esse ataque e aumentar a eficiência de processamento, à custa de algum armazenamento, a multiplicação pode ser substituída por uma pesquisa em tabela. Defina a tabela de 256 bytes $X2$, de modo que $X2[i] = \{02\} \cdot i$. Então, o conjunto de equações 5.9 pode ser reescrito como

$$\begin{aligned}Tmp &= s_{0,j} \oplus s_{1,j} \oplus s_{2,j} \oplus s_{3,j} \\s_{0,j} &= s_{0,j} \oplus Tmp \oplus X2[s_{0,j} \oplus s_{1,j}] \\s_{1,c} &= s_{1,j} \oplus Tmp \oplus X2[s_{1,j} \oplus s_{2,j}] \\s_{2,c} &= s_{2,j} \oplus Tmp \oplus X2[s_{2,j} \oplus s_{3,j}] \\s_{3,j} &= s_{3,j} \oplus Tmp \oplus X2[s_{3,j} \oplus s_{0,j}]\end{aligned}$$

PROCESSADOR DE 32 BITS A implementação descrita na subseção anterior usa apenas operações de 8 bits. Para um processador de 32 bits, uma implementação mais eficiente pode ser conseguida se as operações forem definidas sobre words de 32 bits. Para mostrar isso, primeiro definimos as quatro transformações de uma rodada em formato algébrico. Suponha que comecemos com uma matriz **Estado** consistindo em elementos $a_{i,j}$ e uma matriz de chave de rodada constituída de elementos $k_{i,j}$. Então, as transformações podem ser expressas da seguinte forma:

SubBytes	$b_{i,j} = S[a_{i,j}]$
ShiftRows	$\begin{bmatrix} c_{0,j} \\ c_{1,j} \\ c_{2,j} \\ c_{3,j} \end{bmatrix} = \begin{bmatrix} b_{0,j} \\ b_{1,j-1} \\ b_{2,j-2} \\ b_{3,j-3} \end{bmatrix}$
MixColumns	$\begin{bmatrix} d_{0,j} \\ d_{1,j} \\ d_{2,j} \\ d_{3,j} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} c_{0,j} \\ c_{1,j} \\ c_{2,j} \\ c_{3,j} \end{bmatrix}$
AddRoundKey	$\begin{bmatrix} e_{0,j} \\ e_{1,j} \\ e_{2,j} \\ e_{3,j} \end{bmatrix} = \begin{bmatrix} d_{0,j} \\ d_{1,j} \\ d_{2,j} \\ d_{3,j} \end{bmatrix} \oplus \begin{bmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{bmatrix}$

Na equação de ShiftRows, os índices de coluna são considerados mod 4. Podemos combinar todas essas expressões em uma única equação:

$$\begin{aligned}\begin{bmatrix} e_{0,j} \\ e_{1,j} \\ e_{2,j} \\ e_{3,j} \end{bmatrix} &= \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} S[a_{0,j}] \\ S[a_{1,j-1}] \\ S[a_{2,j-2}] \\ S[a_{3,j-3}] \end{bmatrix} \oplus \begin{bmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{bmatrix} \\ &= \left(\begin{bmatrix} 02 \\ 01 \\ 01 \\ 03 \end{bmatrix} S[a_{0,j}] \right) \oplus \left(\begin{bmatrix} 03 \\ 02 \\ 01 \\ 01 \end{bmatrix} S[a_{1,j-1}] \right) \oplus \left(\begin{bmatrix} 01 \\ 03 \\ 02 \\ 01 \end{bmatrix} S[a_{2,j-2}] \right) \\ &\quad \oplus \left(\begin{bmatrix} 01 \\ 01 \\ 03 \\ 02 \end{bmatrix} S[a_{3,j-3}] \right) \oplus \begin{bmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{bmatrix}\end{aligned}$$

Na segunda equação, estamos expressando a multiplicação de matrizes como uma combinação linear de vetores. Definimos quatro tabelas de 256 words (1024 bytes) da seguinte forma:

$$T_0[x] = \begin{pmatrix} 02 \\ 01 \\ 01 \\ 03 \end{pmatrix} S[x] \quad T_1[x] = \begin{pmatrix} 03 \\ 02 \\ 01 \\ 01 \end{pmatrix} S[x] \quad T_2[x] = \begin{pmatrix} 01 \\ 03 \\ 02 \\ 01 \end{pmatrix} S[x] \quad T_3[x] = \begin{pmatrix} 01 \\ 01 \\ 03 \\ 02 \end{pmatrix} S[x]$$

Assim, cada tabela utiliza um valor de byte como entrada e produz um vetor de coluna (uma word de 32 bits) que é uma função da entrada da S-box para esse valor de byte. Essas tabelas podem ser calculadas antecipadamente.

Podemos definir uma função de rodada operando sobre uma coluna da seguinte forma:

$$\begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix} = T_0[s_{0,j}] \oplus T_1[s_{1,j-1}] \oplus T_2[s_{2,j-2}] \oplus T_3[s_{3,j-3}] \oplus \begin{bmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{bmatrix}$$

Como resultado, uma implementação baseada na equação anterior requer apenas quatro pesquisas em tabela e quatro XORs por coluna por rodada, mais 4 Kbytes para armazenar a tabela. Os desenvolvedores do Rijndael acreditam que essa implementação compacta e eficiente provavelmente foi um dos fatores mais importantes na seleção desse método para o AES.

5.7 LEITURA RECOMENDADA

A descrição mais completa do AES disponível até agora está no livro elaborado pelos desenvolvedores do AES, [DAEM02]. Esses autores também oferecem uma rápida descrição e raciocínio de projeto em [DAEM01]. [LAND04] é um tratado matemático rigoroso do AES e de sua criptoanálise.

Outro exemplo explicado de operação do AES, de autoria dos instrutores na Massey U., Nova Zelândia, está disponível na nossa Sala Virtual, <sv.pearson.com.br>.

DAEM01 Daemen, J. e Rijmen, V. “Rijndael: The Advanced Encryption Standard”. *Dr. Dobb’s Journal*, março de 2001.

DAEM02 Daemen, J. e Rijmen, V. *The Design of Rijndael: The Wide Trail Strategy Explained*. Nova York, Springer-Verlag, 2002.

LAND04 Landau, S. “Polynomials in the Nation’s Service: Using Algebra to Design the Advanced Encryption Standard”. *American Mathematical Monthly*, fev 2004.

5.8 PRINCIPAIS TERMOS, PERGUNTAS PARA REVISÃO E PROBLEMAS

Principais termos

Advanced Encryption Standard (AES)
S-box
corpo
corpo finito

efeito avalanche
expansão de chave
National Institute of Standards and
Technology (NIST)

polinômio irredutível
Rijndael

Perguntas para revisão

- 5.1** Qual foi o conjunto original de critérios usados pelo NIST para avaliar as cifras AES candidatas?
- 5.2** Qual foi o conjunto final de critérios usados pelo NIST para avaliar as cifras AES candidatas?

- 5.3 Qual é a diferença entre Rijndael e AES?
- 5.4 Qual é a finalidade do array **Estado**?
- 5.5 Como é construída a S-box?
- 5.6 Descreva rapidamente o estágio SubBytes.
- 5.7 Descreva rapidamente o estágio ShiftRows.
- 5.8 Quantos bytes em **Estado** são afetados por ShiftRows?
- 5.9 Descreva rapidamente o estágio MixColumns.
- 5.10 Descreva rapidamente o estágio AddRoundKey.
- 5.11 Descreva rapidamente o algoritmo de expansão de chave.
- 5.12 Qual é a diferença entre SubBytes e SubWord?
- 5.13 Qual é a diferença entre ShiftRows e RotWord?
- 5.14 Qual é a diferença entre o algoritmo de decifração AES e a cifra inversa equivalente?

Problemas

- 5.1 Na discussão de MixColumns e InvMixColumns, foi dito que
- $$b(x) = a^{-1}(x) \bmod (x^4 + 1)$$
- onde $a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$ e $b(x) = \{0B\}x^3 + \{0D\}x^2 + \{09\}x + \{0E\}$. Mostre que isso é verdadeiro.
- 5.2
- a. O que é $\{01\}^{-1}$ em $GF(2^8)$?
 - b. Verifique a entrada para $\{01\}$ na S-box.
- 5.3 Mostre as primeiras oito words da expansão de chave para uma de 128 bits, todas com zero.
- 5.4 Dado o texto claro $\{000102030405060708090A0B0C0D0E0F\}$ e a chave $\{01010101010101010101010101010101\}$,
- a. Mostre o conteúdo original de **Estado**, exibido como uma matriz 4×4 .
 - b. Mostre o valor de **Estado** após o **AddRoundKey** inicial.
 - c. Mostre o valor de **Estado** após **SubBytes**.
 - d. Mostre o valor de **Estado** após **ShiftRows**.
 - e. Mostre o valor de **Estado** após **MixColumns**.
- 5.5 Verifique a Equação 5.11. Ou seja, mostre que $x^i \bmod (x^4 + 1) = x^{i \bmod 4}$.
- 5.6 Compare AES com DES. Para cada um dos seguintes elementos do DES, indique o elemento comparável no AES ou explique por que ele não é necessário no AES.
- a. XOR do material da subchave com a entrada da função f .
 - b. XOR da saída da função f com a metade esquerda do bloco.
 - c. função f .
 - d. permutação P .
 - e. troca de metades do bloco.
- 5.7 Na subseção sobre aspectos de implementação, mencionamos que o uso de tabelas ajuda a impedir ataques de temporização. Sugira uma técnica alternativa.
- 5.8 Na subseção sobre aspectos de implementação, uma única equação algébrica é desenvolvida para descrever os quatro estágios de uma rodada típica do algoritmo de encriptação. Forneça a equação equivalente para a décima rodada.
- 5.9 Calcule a saída da transformação MixColumns para a seguinte sequência de bytes de entrada "67 89 AB CD". Aplique a transformação InvMixColumns ao resultado obtido para verificar seus cálculos. Altere o primeiro byte da entrada de "67" para "77", realize a transformação MixColumns novamente para a nova entrada e determine quantos bits mudaram na saída.
- Nota:* você pode realizar todos os cálculos à mão ou escrever um programa que dê suporte a eles. Se escolher escrever um programa, ele deverá ser feito inteiramente por você; nesta tarefa, não use bibliotecas ou código fonte de domínio público.
- 5.10 Use a chave 1010 0111 0011 1011 para encriptar o texto claro "ok" conforme expresso em ASCII, ou seja, 0110 1111 0110 1011. Os projetistas do S-AES obtiveram o texto cifrado 0000 0111 0011 1000. E você?
- 5.11 Mostre que a matriz a seguir, com entradas em $GF(2^4)$, é o inverso daquela usada na etapa MixColumns do S-AES.
- $$\begin{pmatrix} x^3 + 1 & x \\ x & x^3 + 1 \end{pmatrix}$$
- 5.12 Escreva cuidadosamente uma decifração completa do texto cifrado 0000 0111 0011 1000, usando a chave 1010 0111 0011 1011 e o algoritmo S-AES. Você deverá obter o texto claro com que começamos no Problema 5.10. Observe que o inverso das S-boxes pode ser feito com uma pesquisa em tabela reversa. O inverso da etapa MixColumns é dado pela matriz no problema anterior.
- 5.13 Demonstre que a Equação 5.9 é equivalente à Equação 5.4.

Problemas de programação

- 5.14** Crie um software que possa encriptar e decriptar usando S-AES. *Dados de teste:* um texto claro binário de 0110 1111 0110 1011 encriptado com uma chave binária de 1010 0111 0011 1011 deverá dar o texto cifrado binário 0000 0111 0011 1000. A decriptação deverá funcionar da mesma forma.
- 5.15** Implemente um ataque de criptoanálise diferencial no S-AES em uma rodada.

APÊNDICE 5A POLINÔMIOS COM COEFICIENTES EM GF(2⁸)

Na Seção 4.5, discutimos sobre aritmética de polinômio, em que os coeficientes estão em $\mathbb{Z}_p$ e os polinômios são definidos módulo um polinômio $M(x)$, cuja potência mais alta é algum inteiro n . Nesse caso, adição e multiplicação de coeficientes ocorriam dentro do corpo $\mathbb{Z}_p$; ou seja, adição e multiplicação eram realizadas módulo p .

O documento do AES define a aritmética de polinômio para aqueles de grau 3 ou menos com coeficientes em GF(2⁸). As regras a seguir se aplicam:

1. A adição é realizada somando-se os coeficientes correspondentes em GF(2⁸). Conforme indicado na Seção 4.5, se tratarmos os elementos de GF(2⁸) como strings de 8 bits, então a adição é equivalente à operação XOR. Assim, se tivermos

$$a(x) = a_3x^3 + a_2x^2 + a_1x + a_0 \quad (5.10)$$

e

$$b(x) = b_3x^3 + b_2x^2 + b_1x + b_0 \quad (5.11)$$

então

$$a(x) + b(x) = (a_3 \oplus b_3)x^3 + (a_2 \oplus b_2)x^2 + (a_1 \oplus b_1)x + (a_0 \oplus b_0)$$

2. A multiplicação é realizada como a de polinômios normal, com dois refinamentos:
 - a. Os coeficientes são multiplicados em GF(2⁸).
 - b. O polinômio resultante é reduzido mod ($x^4 + 1$).

Precisamos esclarecer de qual polinômio estamos falando. Lembre-se, da Seção 4.6, que cada elemento de GF(2⁸) é um polinômio de grau 7 ou menos com coeficientes binários, e a multiplicação é executada módulo um polinômio de grau 8. De modo equivalente, cada elemento de GF(2⁸) pode ser visto como um byte de 8 bits cujos valores de bits equiparam aos coeficientes binários do polinômio correspondente. Para os conjuntos definidos nesta seção, estamos determinando um anel polinomial em que cada elemento é um polinômio de grau 3 ou menos, com coeficientes em GF(2⁸), e a multiplicação é executada módulo um polinômio de grau 4. De modo equivalente, cada elemento desse anel pode ser visto como uma word de 4 bytes cujos valores de byte são elementos de GF(2⁸) que se relacionam aos coeficientes de 8 bits do polinômio correspondente.

Indicamos o produto modular de $a(x)$ e $b(x)$ com $a(x) \otimes b(x)$. Para calcular $d(x) = a(x) \otimes b(x)$, o primeiro passo é realizar uma multiplicação sem a operação de módulo e coletar coeficientes de potências semelhantes. Vamos expressar isso como $c(x) = a(x) \times b(x)$. Então,

$$c(x) = c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x + c_0 \quad (5.12)$$

onde

$$\begin{aligned} c_0 &= a_0 \ b_0 & c_4 &= (a_3 \ b_1) \oplus (a_2 \ b_2) \oplus (a_1 \ b_3) \\ c_1 &= (a_1 \ b_0) \oplus (a_0 \ b_1) & c_5 &= (a_3 \ b_2) \oplus (a_2 \ b_3) \\ c_2 &= (a_2 \ b_0) \oplus (a_1 \ b_1) \oplus (a_0 \ b_2) & c_6 &= a_3 \ b_3 \\ c_3 &= (a_3 \ b_0) \oplus (a_2 \ b_1) \oplus (a_1 \ b_2) \oplus (a_0 \ b_3) \end{aligned}$$

A última etapa é realizar a operação de módulo:

$$d(x) = c(x) \bmod (x^4 + 1)$$

Ou seja, $d(x)$ precisa satisfazer a equação

$$c(x) = [(x^4 + 1) \times q(x)] \oplus d(x)$$

de modo que o grau de $d(x)$ seja 3 ou menos.

Uma técnica prática para realizar a multiplicação por esse anel polinomial é baseada na observação de que

$$x^i \bmod (x^4 + 1) = x^{i \bmod 4} \quad (5.13)$$

Se agora combinarmos as equações 5.12 e 5.13, acabaremos com

$$\begin{aligned} d(x) &= c(x) \bmod (x^4 + 1) \\ &= [c_6x^6 + c_5x^5 + c_4x^4 + c_3x^3 + c_2x^2 + c_1x + c_0] \bmod (x^4 + 1) \\ &= c_3x^3 + (c_2 \oplus c_6)x^2 + (c_1 \oplus c_5)x + (c_0 \oplus c_4) \end{aligned}$$

Expandindo os coeficientes e_i , temos as seguintes equações para os de $d(x)$:

$$\begin{aligned} d_0 &= (a_0 \ b_0) \oplus (a_3 \ b_1) \oplus (a_2 \ b_2) \oplus (a_1 \ b_3) \\ d_1 &= (a_1 \ b_0) \oplus (a_0 \ b_1) \oplus (a_3 \ b_2) \oplus (a_2 \ b_3) \\ d_2 &= (a_2 \ b_0) \oplus (a_1 \ b_1) \oplus (a_0 \ b_2) \oplus (a_3 \ b_3) \\ d_3 &= (a_3 \ b_0) \oplus (a_2 \ b_1) \oplus (a_1 \ b_2) \oplus (a_0 \ b_3) \end{aligned}$$

Isso pode ser escrito em forma de matriz:

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} \quad (5.14)$$

Transformação MixColumns

Na discussão sobre MixColumns, foi dito que existem duas maneiras equivalentes de definir a transformação. A primeira é a multiplicação matricial, mostrada na Equação 5.3 e repetida aqui:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

O segundo método é tratar cada coluna de **Estado** como um polinômio de quatro termos com coeficientes em $\text{GF}(2^8)$. Cada coluna é multiplicada módulo $(x^4 + 1)$ pelo polinômio fixo $a(x)$, dado por

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$

Pela Equação 5.10, temos $a_3 = \{03\}$; $a_2 = \{01\}$; $a_1 = \{01\}$; $a_0 = \{02\}$. Para a coluna j de **Estado**, temos o polinômio $\text{col}_j(x) = s_{3,j}x^3 + s_{2,j}x^2 + s_{1,j}x + s_{0,j}$. Substituindo na Equação 5.14, podemos expressar $d(x) = a(x) \times \text{col}_j(x)$ como

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{bmatrix} \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix}$$

que é equivalente à Equação 5.3.

Multiplicação por x

Considere a multiplicação de um polinômio no anel por x : $c(x) = x \oplus b(x)$. Temos

$$\begin{aligned}
 c(x) &= x \otimes b(x) = [x \times (b_3x^3 + b_2x^2 + b_1x + b_0)] \bmod (x^4 + 1) \\
 &= (b_3x^4 + b_2x^3 + b_1x^2 + b_0x) \bmod (x^4 + 1) \\
 &= b_2x^3 + b_1x^2 + b_0x + b_3
 \end{aligned}$$

Assim, a multiplicação por x corresponde a um deslocamento circular à esquerda por 1 byte dos 4 bytes na word que representa o polinômio. Se sinalizarmos o polinômio como um vetor de colunas de 4 bytes, então temos

$$\begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} 00 & 00 & 00 & 01 \\ 01 & 00 & 00 & 00 \\ 00 & 01 & 00 & 00 \\ 00 & 00 & 01 & 00 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

APÊNDICE 5B AES SIMPLIFICADO

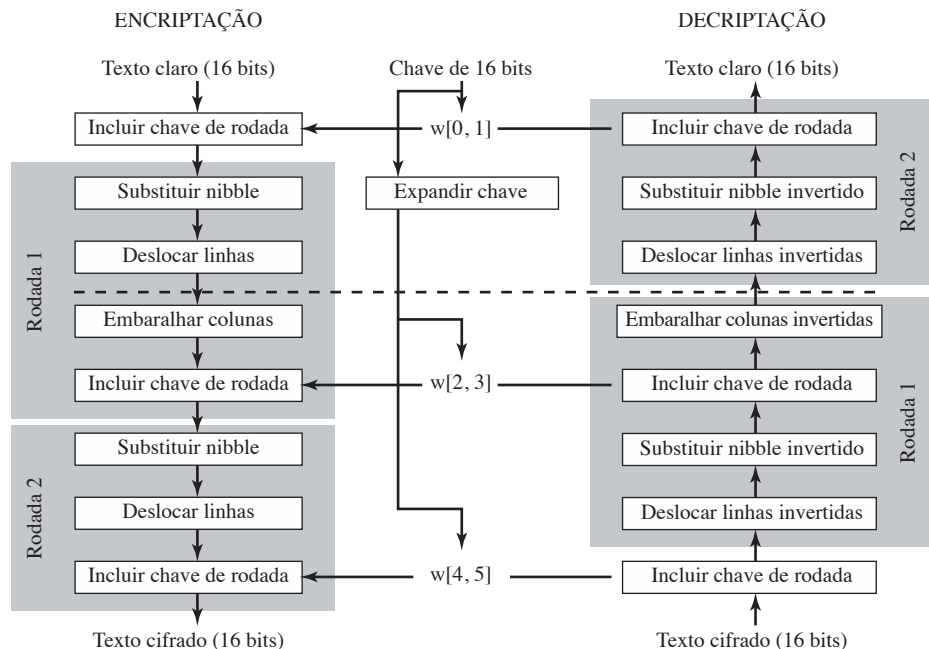
O AES simplificado (S-AES) foi desenvolvido pelo Professor Edward Schaefer da Santa Clara University e por vários de seus alunos [MUSA03]. Ele é um algoritmo de encriptação educacional, em vez de um seguro. Ele possui propriedades e estrutura semelhantes ao AES, com parâmetros muito menores. O leitor poderá achar útil trabalhar com um exemplo à mão enquanto acompanha a discussão neste apêndice. Um bom conhecimento do S-AES facilitará a compreensão da estrutura e do funcionamento do AES pelo aluno.

Visão geral

A Figura 5.11 ilustra a estrutura geral do S-AES. O algoritmo de encriptação utiliza um bloco de 16 bits de texto claro como entrada e uma chave de 16 bits, e produz um bloco de 16 bits de texto cifrado como saída. O algoritmo de deciptação do S-AES emprega um bloco de 16 bits de texto cifrado e a mesma chave de 16 bits usada para produzir esse texto cifrado como entrada, criando como saída o bloco de 16 bits de texto claro original.

O algoritmo de encriptação envolve o uso de quatro funções diferentes, ou transformações: incluir chave (A_K), substituir nibble (NS), deslocar linha (SR) e embaralhar coluna (MC), cuja operação é explicada mais adiante.

Figura 5.11 Encriptação e deciptação do S-AES.



Podemos expressar o algoritmo de encriptação de forma concisa como uma composição⁶ de funções:

$$A_{K_2} \text{ SR NS } A_{K_1} \text{ MC SR NS } A_{K_0}$$

de modo que A_{K_0} é aplicado primeiro.

O algoritmo de encriptação é organizado em três rodadas. A rodada 0 é simplesmente de inclusão de chave; a rodada 1 é completa de quatro funções; e a rodada 2 contém apenas três funções. Cada rodada conta com a função incluir chave, que utiliza os 16 bits dela. A chave inicial de 16 bits é expandida para 48 bits, de modo que cada rodada utiliza uma chave de rodada distinta de 16 bits.

Cada função opera sobre um estado de 16 bits, tratado como uma matriz 2×2 de nibbles, na qual um nibble é igual a 4 bits. O valor inicial da matriz **Estado** é o texto claro de 16 bits; ela é modificada por cada função subsequente no processo de encriptação, produzindo após a última o texto cifrado de 16 bits. Como mostra a Figura 5.12a, a ordenação dos nibbles dentro da matriz é por coluna. Assim, por exemplo, os primeiros oito bits de uma entrada de texto claro de 16 bits para a cifra de encriptação ocupam a primeira coluna da matriz, e os próximos oito bits, a segunda coluna. A chave de 16 bits é organizada de modo semelhante, mas é um pouco mais conveniente vê-la como dois bytes, em vez de quatro nibbles (Figura 5.12b). A chave expandida de 48 bits é tratada como três chaves de rodada, cujos bits são rotulados da seguinte forma: $K_0 = k_0 \dots k_{15}$; $K_1 = k_{16} \dots k_{31}$; $K_2 = k_{32} \dots k_{47}$.

A Figura 5.13 mostra os elementos essenciais de uma rodada completa do S-AES.

Figura 5.12 Estruturas de dados do S-AES.

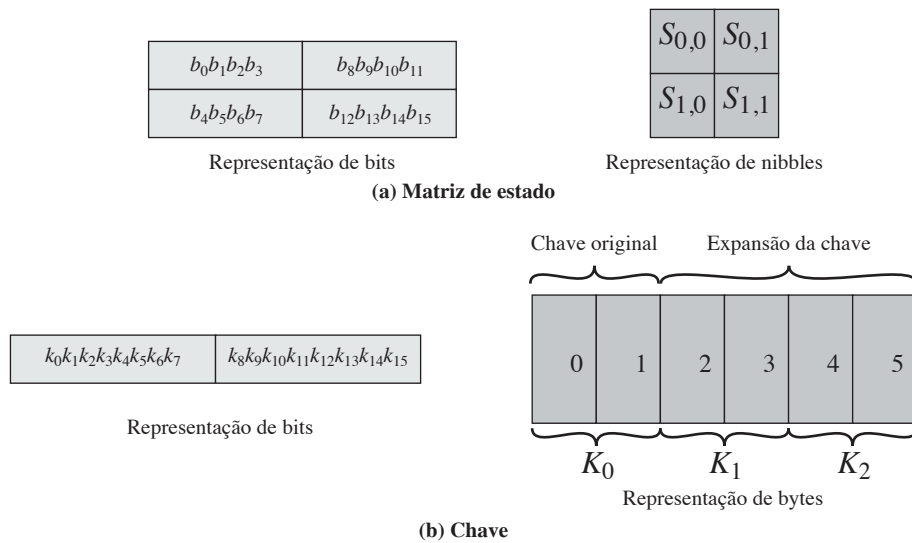
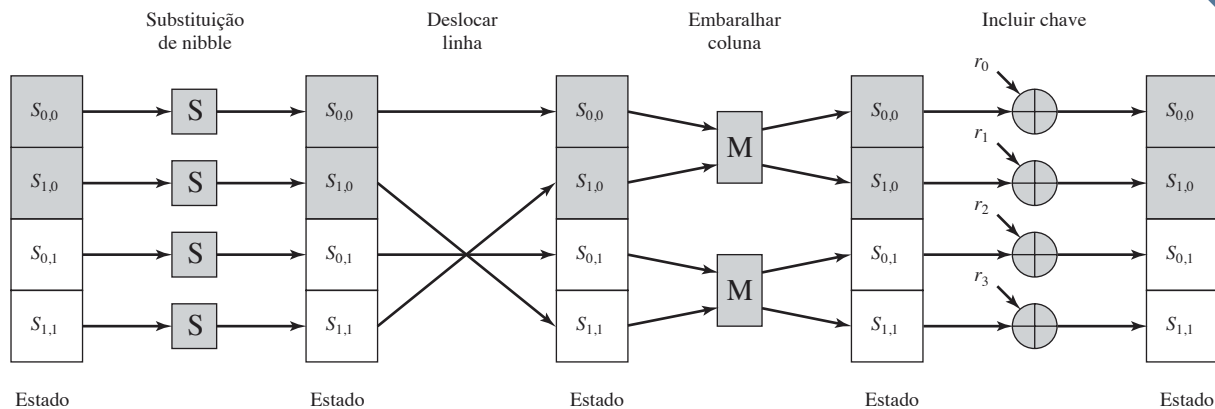


Figura 5.13 Rodada de encriptação do S-AES.



⁶ **Definição:** se f e g são duas funções, então a F com a equação $y = F(x) = g[f(x)]$ é chamada de **composição** de f e g , sendo indicada como $F = g \circ f$.

A decriptação também aparece na Figura 5.11 e é basicamente o inverso da encriptação:

$$A_{K_0} \text{ INS ISR IMC } A_{K_1} \text{ INS ISR } A_{K_2}$$

em que três das funções têm uma inversa correspondente: substituir nibbles invertidos (INS), substituir linhas invertidas (ISR) e embaralhar colunas invertidas (IMC).

Encriptação e decriptação S-AES

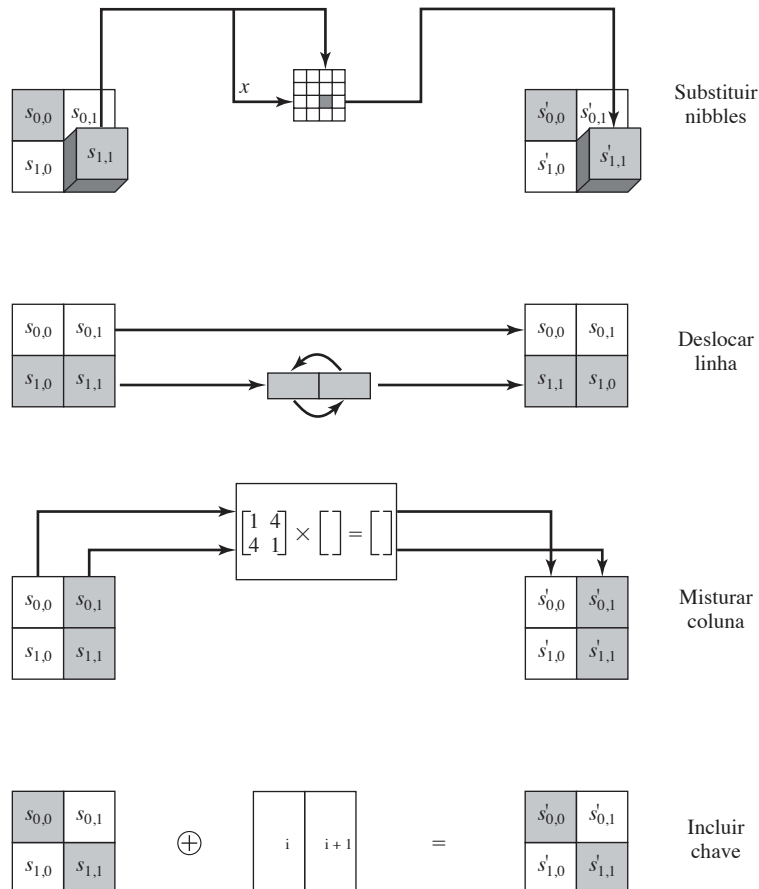
Agora, vejamos as funções individuais que fazem parte do algoritmo de encriptação.

INCLUSÃO DE CHAVE A função incluir chave consiste no XOR bit a bit entre a matriz **Estado** de 16 bits e a chave de rodada de 16 bits. A Figura 5.14 representa isso como uma operação coluna por coluna, mas também pode ser visto como uma operação nibble a nibble ou bit a bit. Veja um exemplo a seguir:

<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td style="padding: 5px;">A</td><td style="padding: 5px;">4</td></tr> <tr><td style="padding: 5px;">7</td><td style="padding: 5px;">9</td></tr> </table> Matriz estado	A	4	7	9	$\oplus$	<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td style="padding: 5px;">2</td><td style="padding: 5px;">5</td></tr> <tr><td style="padding: 5px;">D</td><td style="padding: 5px;">5</td></tr> </table> Chave	2	5	D	5	$=$	<table border="1" style="border-collapse: collapse; text-align: center;"> <tr><td style="padding: 5px;">8</td><td style="padding: 5px;">1</td></tr> <tr><td style="padding: 5px;">A</td><td style="padding: 5px;">C</td></tr> </table>	8	1	A	C
A	4															
7	9															
2	5															
D	5															
8	1															
A	C															

O inverso da função incluir chave é idêntico à função adicionar chave, pois a operação XOR é o seu próprio inverso.

Figura 5.14 Transformações S-AES.



Substituição de nibble

A função de substituição de nibble é uma simples pesquisa em tabela (Figura 5.14). O AES define uma matriz 4×4 de valores de nibble, chamada S-box (Tabela 5.7a), que contém uma permutação de todos os valores de 4 bits possíveis. Cada nibble individual da matriz **Estado** é comparada com um novo nibble da seguinte maneira: os 2 bits mais à esquerda do nibble são usados como um valor de linha, e os 2 bits mais à direita, como um valor de coluna. Esses valores de linha e coluna servem como índices na S-box, para selecionar um valor único de saída de 4 bits. Por exemplo, o valor hexadecimal A referencia a linha 2, coluna 2 da S-box, que contém o valor 0. Por conseguinte, o valor A é comparado com o valor 0.

Aqui está um exemplo da transformação de substituição de nibble:

8	1
A	C

→

6	4
C	0

A função de substituição de nibble inversa utiliza a S-box invertida, mostrada na Tabela 5.7b. Observe, por exemplo, que a entrada 0 produz a saída A, e a saída A para a S-box produz 0.

Deslocamento de linha

A função deslocar linhas realiza um deslocamento circular de um nibble da segunda linha da matriz de **Estado**; a primeira linha não é alterada (Figura 5.14). A seguir vemos um exemplo:

6	4
0	C

→

6	4
C	0

A função deslocar linhas invertidas é idêntica à função deslocar linhas, pois desloca a segunda linha de volta à sua posição original.

Embaralhar colunas

A função embaralhar colunas opera sobre cada coluna individualmente. Cada nibble de uma coluna é mapeado para um novo valor que é uma função de ambos os nibbles nessa coluna. A transformação pode ser definida pela seguinte multiplicação sobre a matriz de **estado** (Figura 5.14).

$$\begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} \\ s_{1,0} & s_{1,1} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} \\ s_{1,0} & s_{1,1} \end{bmatrix}$$

Realizando a multiplicação de matriz, obtemos:

$$S_{0,0} = S_{0,0} \oplus (4 \cdot S_{1,0})$$

$$S_{1,0} = (4 \cdot S_{0,0}) \oplus S_{1,0}$$

$$S_{0,1} = S_{0,1} \oplus (4 \cdot S_{1,1})$$

$$S_{1,1} = (4 \cdot S_{0,1}) \oplus S_{1,1}$$

onde a aritmética é realizada em $GF(2^4)$, e o símbolo $\cdot$ refere-se à multiplicação em $GF(2^4)$. O Apêndice I (<sv.pearson.com.br>, em inglês) oferece as tabelas de adição e multiplicação. A seguir vemos um exemplo:

Tabela 5.7 S-boxes do S-AES.

		<i>j</i>			
		00	01	10	11
<i>i</i>	00	9	4	A	B
	01	D	1	8	5
	10	6	2	0	3
	11	C	E	F	7

(a) S-box

		<i>j</i>			
		00	01	10	11
<i>i</i>	00	A	5	9	B
	01	1	7	8	F
	10	6	0	2	3
	11	C	4	D	E

(b) S-box invertida

Nota: números hexadecimais em caixas sombreadas; números binários em caixas não sombreadas.

$$\begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} 6 & 4 \\ C & 0 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 7 & 3 \end{bmatrix}$$

A função embaralhar coluna invertida é definida da seguinte forma:

$$\begin{bmatrix} 9 & 2 \\ 2 & 9 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} \\ s_{1,0} & s_{1,1} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} \\ s_{1,0} & s_{1,1} \end{bmatrix}$$

Demonstramos que de fato definimos o inverso:

$$\begin{bmatrix} 9 & 2 \\ 2 & 9 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} \\ s_{1,0} & s_{1,1} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} \\ s_{1,0} & s_{1,1} \end{bmatrix} = \begin{bmatrix} s_{0,0} & s_{0,1} \\ s_{1,0} & s_{1,1} \end{bmatrix}$$

A multiplicação matricial anterior utiliza os seguintes resultados em $GF(2^4)$: $9 + (2 \cdot 4) = 9 + 8 = 1$ e $(9 \cdot 4) + 2 = 2 + 2 = 0$. Essas operações podem ser verificadas por meio de tabelas aritméticas no Apêndice I ou pela aritmética de polinômios.

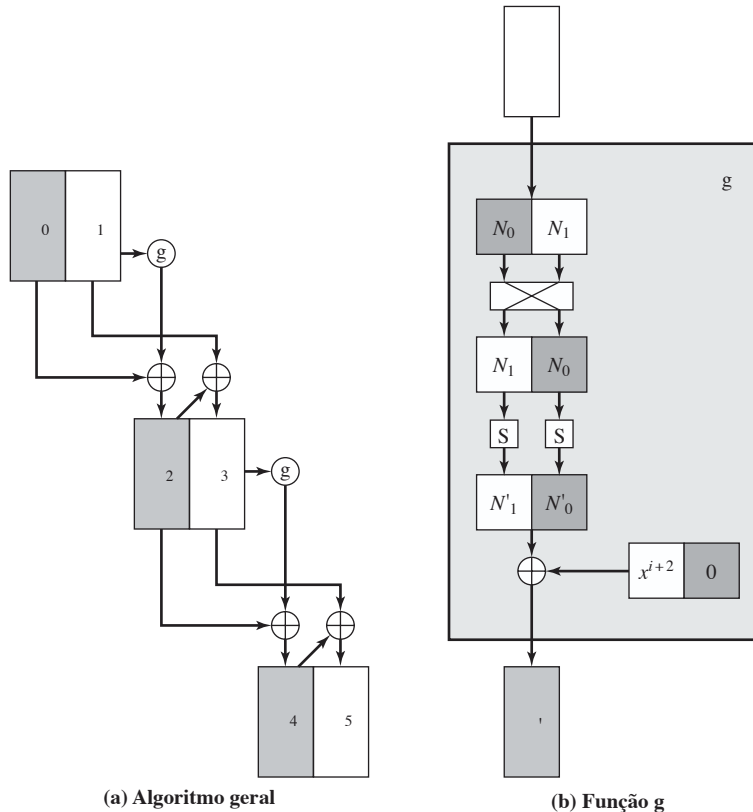
A função de embaralhamento de colunas é a mais difícil de se visualizar. Por conseguinte, oferecemos uma perspectiva adicional para ela no Apêndice I.

Expansão de chave

Para a expansão de chave, os 16 bits da chave inicial são agrupados em uma fileira de duas words de 8 bits. A Figura 5.15 mostra a expansão para 6 words, pelo cálculo de 4 novas words a partir das 2 words iniciais. O algoritmo é o seguinte:

$$\begin{aligned} w_2 &= w_0 \oplus g(w_1) = w_0 \oplus \text{Rcon}(1) \oplus \text{SubNib}(\text{RotNib}(w_1)) \\ w_3 &= w_2 \oplus w_1 \\ w_4 &= w_2 \oplus g(w_3) = w_2 \oplus \text{Rcon}(2) \oplus \text{SubNib}(\text{RotNib}(w_3)) \\ w_5 &= w_4 \oplus w_3 \end{aligned}$$

Figura 5.15 Expansão de chave do S-AES.



Rcon é uma constante de rodada, definida da seguinte forma: $RC[i] = x^{i+2}$, de modo que $RC[1] = x^3 = 1000$, e $RC[2] = x^4 \bmod (x^4 + x + 1) = x + 1 = 0011$. $RC[i]$ forma o nibble mais à esquerda de um byte, com o nibble mais à direita sendo composto apenas de zeros. Assim, $Rcon(1) = 10000000$ e $Rcon(2) = 00110000$.

Por exemplo, suponha que a chave seja $2D55 = 0010\ 1101\ 0101\ 0101 = w_0w_1$. Então

$$\begin{aligned} w_2 &= 00101101 \oplus 10000000 \oplus \text{SubNib}(01010101) \\ &= 00101101 \oplus 10000000 \oplus 00010001 = 10111100 \\ w_3 &= 10111100 \oplus 01010101 = 11101001 \\ w_4 &= 10111100 \oplus 00110000 \oplus \text{SubNib}(10011110) \\ &= 10111100 \oplus 00110000 \oplus 00101111 = 10100011 \\ w_5 &= 10100011 \oplus 11101001 = 01001010 \end{aligned}$$

A S-box

A S-box é construída da seguinte forma:

1. Inicialize a caixa-S com os valores de nibble em sequência crescente linha por linha. A primeira linha contém os valores hexadecimais (0, 1, 2, 3); a segunda linha, (4, 5, 6, 7); e assim por diante. Dessa forma, o valor do nibble na linha i , coluna j é $4i + j$.
2. Trate cada nibble como um elemento do corpo finito $GF(2^4)$ módulo $x^4 + x + 1$. Cada nibble $a_0 a_1 a_2 a_3$ representa um polinômio de grau 3.
3. Compare cada byte da S-box com seu inverso multiplicativo no corpo finito $GF(2^4)$ módulo $x^4 + x + 1$; o valor 0 é comparado consigo mesmo.
4. Considere que cada byte na S-box consiste em 4 bits rotulados com (b_0, b_1, b_2, b_3) . Aplique a transformação a seguir a cada bit de cada byte na S-box. O padrão do AES representa essa transformação no formato de matriz da seguinte maneira:

$$\begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

5. Aspas simples (') indicam que a variável deve ser atualizada pelo valor à direita. Lembre-se de que adição e multiplicação estão sendo calculadas com módulo 2.

A Tabela 5.7a mostra a S-box resultante. Esta é uma matriz não linear e reversível. A S-box inversa aparece na Tabela 5.7b.

Estrutura do S-AES

Agora, podemos examinar vários aspectos com relação à estrutura do AES. Primeiro, observe que os algoritmos de encriptação e decríptação começam e terminam com a função incluir chave. Qualquer outra função, no início ou no fim, é facilmente reversível sem conhecimento da chave, e por isso não incluiria segurança, mas apenas um overhead de processamento. Assim, existe uma rodada 0 consistindo apenas na função incluir chave.

O segundo ponto a observar é que a rodada 2 não inclui a função embaralhar coluna. A explicação para isso na verdade se relaciona com uma terceira observação, de que, embora o algoritmo de decríptação seja o reverso do de encriptação, como podemos ver claramente na Figura 5.11, ele não acompanha a mesma sequência de funções. Assim,

$$\begin{aligned} \text{Encriptação: } & A_{K_2} \circ SR \circ NS \circ A_{K_1} \circ MC \circ SR \circ NS \circ A_{K_0} \\ \text{Decríptação: } & A_{K_0} \circ INS \circ ISR \circ IMC \circ A_{K_1} \circ INS \circ ISR \circ A_{K_2} \end{aligned}$$

Por um ponto de vista da implementação, seria desejável que a função de decríptação seguisse a mesma sequência de função da encriptação. Isso permite que o algoritmo de decríptação seja implementado da mesma maneira que o algoritmo de encriptação, criando oportunidades para um modelo mais eficiente.

Observe que, se pudéssemos trocar a segunda e a terceira funções, a quarta e a quinta funções, e a sexta e a sétima funções na sequência de deciptação, teríamos a mesma estrutura do algoritmo de encriptação. Vejamos se isso é possível. Primeiro, considere a troca de INS e ISR. Dado um estado N consistindo nos nibbles (N_0, N_1, N_2, N_3) , a transformação $\text{INS}(\text{ISR}(N))$ prossegue da seguinte forma:

$$\begin{pmatrix} N_0 & N_2 \\ N_1 & N_3 \end{pmatrix} \rightarrow \begin{pmatrix} N_0 & N_2 \\ N_3 & N_1 \end{pmatrix} \rightarrow \begin{pmatrix} \text{IS}[N_0] & \text{IS}[N_2] \\ \text{IS}[N_3] & \text{IS}[N_1] \end{pmatrix}$$

onde IS refere-se à S-box inversa. Revertendo as operações, a transformação $\text{ISR}(\text{INS}(N))$ prossegue da seguinte forma:

$$\begin{pmatrix} N_0 & N_2 \\ N_1 & N_3 \end{pmatrix} \rightarrow \begin{pmatrix} \text{IS}[N_0] & \text{IS}[N_2] \\ \text{IS}[N_1] & \text{IS}[N_3] \end{pmatrix} \rightarrow \begin{pmatrix} \text{IS}[N_0] & \text{IS}[N_2] \\ \text{IS}[N_3] & \text{IS}[N_1] \end{pmatrix}$$

que é o mesmo resultado. Assim, $\text{INS}(\text{ISR}(N)) = \text{ISR}(\text{INS}(N))$.

Agora, considere a operação embaralhar colunas invertidas, seguida por incluir chave: $\text{IMC}(A_{K_1}(N))$, onde a chave da rodada K_1 consiste nos nibbles $(k_{0,0}, k_{1,0}, k_{0,1}, k_{1,1})$. Então:

$$\begin{aligned} \begin{pmatrix} 9 & 2 \\ 2 & 9 \end{pmatrix} \left(\begin{pmatrix} k_{0,0} & k_{0,1} \\ k_{1,0} & k_{1,1} \end{pmatrix} \oplus \begin{pmatrix} N_0 & N_2 \\ N_1 & N_3 \end{pmatrix} \right) &= \begin{pmatrix} 9 & 2 \\ 2 & 9 \end{pmatrix} \begin{pmatrix} k_{0,0} \oplus N_0 & k_{0,1} \oplus N_2 \\ k_{1,0} \oplus N_1 & k_{1,1} \oplus N_3 \end{pmatrix} \\ &= \begin{pmatrix} 9(k_{0,0} \oplus N_0) \oplus 2(K_{1,0} \oplus N_1) & 9(k_{0,1} \oplus N_2) \oplus 2(K_{1,1} \oplus N_3) \\ 2(k_{0,0} \oplus N_0) \oplus 9(K_{1,0} \oplus N_1) & 2(k_{0,1} \oplus N_2) \oplus 9(K_{1,1} \oplus N_3) \end{pmatrix} \\ &= \begin{pmatrix} (9k_{0,0} \oplus 2k_{1,0}) \oplus (9N_0 \oplus 2N_1) & (9k_{0,1} \oplus 2k_{1,1}) \oplus (9N_2 \oplus 2N_3) \\ (2k_{0,0} \oplus 9k_{1,0}) \oplus (2N_0 \oplus 9N_1) & (2k_{0,1} \oplus 9k_{1,1}) \oplus (2N_2 \oplus 9N_3) \end{pmatrix} \\ &= \begin{pmatrix} (9k_{0,0} \oplus 2k_{1,0}) & (9k_{0,1} \oplus 2k_{1,1}) \\ (2k_{0,0} \oplus 9k_{1,0}) & (2k_{0,1} \oplus 9k_{1,1}) \end{pmatrix} \oplus \begin{pmatrix} (9N_0 \oplus 2N_1) & (9N_2 \oplus 2N_3) \\ (2N_0 \oplus 9N_1) & (2N_2 \oplus 9N_3) \end{pmatrix} \\ &= \begin{pmatrix} 9 & 2 \\ 2 & 9 \end{pmatrix} \begin{pmatrix} k_{0,0} & k_{0,1} \\ k_{1,0} & k_{1,1} \end{pmatrix} \oplus \begin{pmatrix} 9 & 2 \\ 2 & 9 \end{pmatrix} \begin{pmatrix} N_0 & N_2 \\ N_1 & N_3 \end{pmatrix} \end{aligned}$$

Todas as etapas citadas utilizam as propriedades da aritmética de corpo finito. O resultado é que $\text{IMC}(A_{K_1}(N)) = \text{IMC}(K_1) \oplus \text{IMC}(N)$. Agora, vamos definir a chave de rodada inversa para a rodada 1 como sendo $\text{IMC}(K_1)$, e a operação incluir chave invertida IA_{K_1} como o XOR bit a bit da chave de rodada inversa com o vetor de estado. Então, temos $\text{IMC}(A_{K_1}(N)) = \text{IA}_{K_1}(\text{IMC}(N))$. Como resultado, podemos escrever o seguinte:

$$\begin{aligned} \textbf{Encriptação:} & A_{K_2} \circ \text{SR} \circ \text{NS} \circ A_{K_1} \circ \text{MC} \circ \text{SR} \circ \text{NS} \circ A_{K_0} \\ \textbf{Deciptação:} & A_{K_0} \circ \text{INS} \circ \text{ISR} \circ \text{IMC} \circ A_{K_1} \circ \text{INS} \circ \text{ISR} \circ A_{K_2} \\ \textbf{Deciptação:} & A_{K_0} \circ \text{ISR} \circ \text{INS} \circ A_{\text{IMC}(K_1)} \circ \text{IMC} \circ \text{ISR} \circ \text{INS} \circ A_{K_2} \end{aligned}$$

Encriptação e deciptação agora seguem a mesma sequência. Observe que essa derivação não funcionaria de modo tão eficaz se a rodada 2 do algoritmo de encriptação incluísse a função MC. Nesse caso, teríamos

$$\begin{aligned} \textbf{Encriptação:} & A_{K_2} \circ \text{MC} \circ \text{SR} \circ \text{NS} \circ A_{K_1} \circ \text{MC} \circ \text{SR} \circ \text{NS} \circ A_{K_0} \\ \textbf{Deciptação:} & A_{K_0} \circ \text{INS} \circ \text{ISR} \circ \text{IMC} \circ A_{K_1} \circ \text{INS} \circ \text{ISR} \circ \text{IMC} \circ A_{K_2} \end{aligned}$$

Agora, não existe como trocar os pares de operações no algoritmo de deciptação de modo a alcançar a mesma estrutura do algoritmo de encriptação.